

BEHAVIOR-1K: A Human-Centered, Embodied AI Benchmark with 1,000 Everyday Activities and Realistic Simulation

Chengshu Li^{*1}, Ruohan Zhang^{*1}, Josiah Wong^{*2},
 Cem Gokmen^{*1}, Sanjana Srivastava^{*1}, Roberto Martín-Martín^{*9},
 Chen Wang^{*1}, Gabrael Levine^{*1}, Wensi Ai^{*1},
 Benjamin Martinez¹, Hang Yin¹, Michael Lingelbach³, Minjune Hwang¹,
 Ayano Hiranaka¹, Sujay Garlanka¹¹, Arman Aydin¹, Sharon Lee¹,
 Jiankai Sun⁴, Mona Anvari¹, Manasi Sharma¹, Dhruva Bansal¹,
 Samuel Hunter¹, Kyu-Young Kim¹, Alan Lou⁵, Caleb R Matthews¹,
 Ivan Villa-Renteria¹, Jerry Huayang Tang¹, Claire Tang¹, Fei Xia⁶,
 Yunzhu Li¹⁰, Silvio Savarese^{1,8,12}, Hyowon Gweon^{7,8},
 C. Karen Liu^{1,8}, Jiajun Wu^{1,8}, Li Fei-Fei^{1,8}

Department of Computer Science¹, Department of Mechanical Engineering²
 Neurosciences IDP³, Department of Aeronautics and Astronautics⁴
 Institute for Computational and Mathematical Engineering⁵
 Department of Electrical Engineering⁶, Department of Psychology⁷
 Institute for Human-Centered Artificial Intelligence (HAI)⁸
 Stanford University

The University of Texas at Austin⁹
 The University of Illinois Urbana-Champaign¹⁰
 University of Southern California¹¹
 Salesforce Research¹²

Abstract: We present BEHAVIOR-1K, a comprehensive simulation benchmark for human-centered robotics. BEHAVIOR-1K includes two components, guided and motivated by the results of an extensive survey on ‘*what do you want robots to do for you?*’. The first is the definition of 1,000 everyday activities, grounded in 50 scenes (houses, gardens, restaurants, offices, etc.) with more than 9,000 objects annotated with rich physical and semantic properties. The second is OMNIGIBSON, a novel simulation environment that supports these activities via realistic physics simulation and rendering of rigid bodies, deformable bodies, and liquids. Our experiments indicate that the activities in BEHAVIOR-1K are long-horizon and dependent on complex manipulation skills, both of which remain a challenge for even state-of-the-art robot learning solutions. To calibrate the simulation-to-reality gap of BEHAVIOR-1K, we provide an initial study on transferring solutions learned with a mobile manipulator in a simulated apartment to its real-world counterpart. We hope that BEHAVIOR-1K’s human-grounded nature, diversity, and realism make it valuable for embodied AI and robot learning research. Project website: <https://behavior.stanford.edu>.

Keywords: Embodied AI Benchmark, Everyday Activities, Mobile Manipulation

^{*} indicates equal contribution
 correspondence to {chengshu,zharu,jdwong,cgokmen,sanjana2}@stanford.edu

1 Introduction

Inspired by the progress that benchmarking brought to computer vision [1–11] and natural language processing [12–16], the robotics community has developed several benchmarks in simulation [17–30]. The broader goal of these benchmarks is to fuel the development of general, effective robots that bring major benefits to people’s daily lives—human-centered AI that “serves human needs, goals, and values” [31–34]. Inspiring as they are, the tasks and activities in those benchmarks are designed by researchers; it remains unclear if they are addressing the actual needs of humans.

We observe that a human-centered robotic benchmark should not only be designed *for* human needs, but also originated *from* human needs: *what everyday activities do humans want robots to do for them?* To this end, we conduct an extensive survey with 1,461 participants (see Sec. 2) to rank a wide range of daily activities based on participants’ desire to delegate these activities to robots. We also ask layperson annotators to provide definitions of those activities. The survey reveals systematicity in what activities people want robots to do, but more importantly, highlights two key factors that we should prioritize when designing robotic benchmarks: **diversity** in the type of scenes, objects, and activities, and **realism** of the underlying simulation environments.

The most needed activities indicated by the survey range from ‘wash floor’ to ‘clean bathtub.’ Clearly, the diversity of these activities is far beyond what real-world robotics challenges may offer [35–42]. Developing simulation environments is a natural alternative: one can train and test robotic agents in many activities with diverse scenes, objects, and conditions efficiently and safely. However, for this paradigm to work, the activities have to be simulated realistically, reproducing accurately the circumstances that a robot may encounter in the real world. While significant progress in realism has been made in specific domains [43–45], achieving realism for a diverse set of activities remains a tremendous challenge, due to the effort required to provide realistic models and simulation features.

In this work, we present **BEHAVIOR-1K**, a Benchmark of 1,000 Everyday Household Activities in Virtual, Interactive, and Ecological Environments—the next generation of BEHAVIOR-100 [27]. BEHAVIOR-1K includes two novel components to address the demands for diversity and realism: the diverse **BEHAVIOR-1K DATASET** and the realistic **OMNIGIBSON** simulation environment. The BEHAVIOR-1K DATASET is a large-scale dataset comprising 1) a commonsense knowledge base for 1,000 activities with definitions in predicate logic (initial and goal conditions), as well as the objects involved, their properties, and their state transitions, and 2) high-quality 3D assets including 50 scenes and 9,000+ object models with rich physical and semantic annotations.

All activities in the BEHAVIOR-1K DATASET are instantiated in a novel simulation environment, OMNIGIBSON, which we build on top of Nvidia’s Omniverse and PhysX 5 [46] to provide realistic physics simulation and rendering of rigid bodies, deformable bodies, and fluids. OMNIGIBSON expands beyond Omniverse’s capabilities with a set of extended object states like temperature, toggled, soaked, and dirtiness. It also includes capabilities to generate valid initial activity configurations and discriminate valid goal solutions based on activity definitions. With all these realistic simulation features, OMNIGIBSON supports the 1,000 diverse activities in the BEHAVIOR-1K DATASET.

We evaluate state-of-the-art reinforcement learning algorithms [47, 48] in several activities of BEHAVIOR-1K, both with visuomotor control in the original action space, and with action primitives that leverage sampling-based motion planning [49]. Our analysis indicates that even a single activity in BEHAVIOR-1K is extremely challenging for current AI algorithms, and the baselines can only solve it with a significant injection of domain knowledge. Concretely, the difficulties derive in part from the length of BEHAVIOR-1K’s activities and the complexity of the physical manipulation required. To calibrate the simulation-to-real gap of BEHAVIOR-1K, we provide an initial study on transferring solutions learned with a mobile manipulator in a simulated apartment to its real-world counterpart. We hope that the BEHAVIOR-1K benchmark, our survey, and our analysis will serve to support and guide the development of future embodied AI agents and robots.

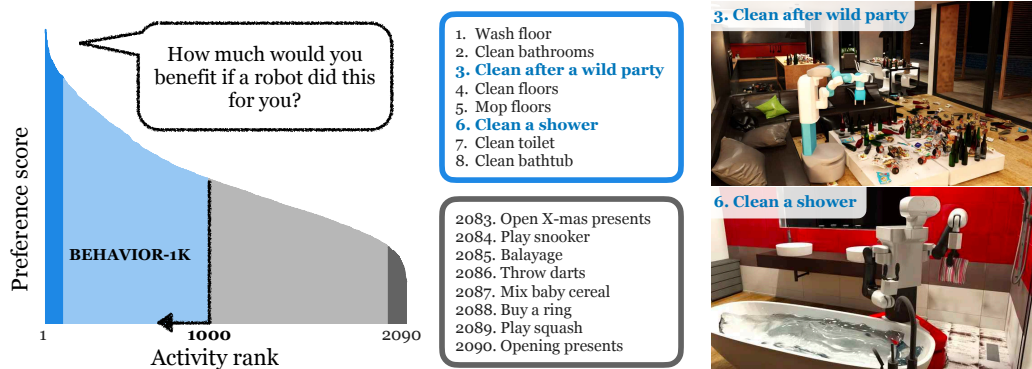


Figure 1: **Developing a Human-Centered Benchmark for Embodied AI.** Left: human preference score over 2,090 activities, ranked based on a survey on 1,461 participants. The distribution indicates the high **diversity** of needs and preferences of humans that should be reflected in a comprehensive benchmark. Middle: Example activities. Laborious activities are ranked the highest, while pleasurable ones are ranked the lowest. Right: visualization of two of the top 8 activities generated by our **realistic** OMNIGIBSON simulation environment.

2 Creating a Benchmark Grounded in Human Needs: A Survey Study

A significant amount of robotics research aspires to satisfy human needs, but those needs are typically assumed or speculated. Human-centric development requires direct information about what humans want from autonomous agents [31]. To create a benchmark that reflects these needs, we conduct a survey targeting the general U.S. population that asks: *what do you want robots to do for you?* The survey sources around 2,000 activities from time-use surveys [50–52], which record how people spend their time, and from WikiHow articles [53]. We conduct the survey on Amazon Mechanical Turk with a total of 1,461 respondents (demographics in Appendix A.3) and fifty 10-point Likert scale responses per activity.

Survey results are summarized in Fig. 1 (left), in which we rank the activities based on their human preference score. The full list of ranked activities can be found on our website. The distribution shows large statistical dispersion (Gini index= 0.158): humans want robots to perform a wide range of activities, from cleaning chores to cooking large feasts. Tedious tasks like “scrubbing the bathroom floor” score the highest, while recreational activities like game-play score the lowest. There are around 200 cleaning activities and over 200 cooking activities, among many other categories.

BEHAVIOR-1K activities include the 909 activities with the highest human preference scores and 91 activities from BEHAVIOR-100 [27], which total the top-ranked 1,000 activities. BEHAVIOR-1K sets itself apart from other embodied AI benchmarks by being sourced from time-use surveys and using survey data to prioritize activities considered most important and useful by humans, and by including a tremendously diverse set of activities.

3 Related Work: Embodied AI Benchmarks

We provide an extensive comparison between BEHAVIOR-1K, and other embodied AI benchmarks in simulation [17–26] in Table 1. We include a number of factors that contribute to diversity and realism and observe a significant step forward with BEHAVIOR-1K. First, no other benchmark grounds their activity set on the needs of lay people. Other benchmarks often target a relatively restricted set of activities, and their simulators are realistic only in the relevant aspects for those tasks. In fact, we often observe a diversity-realism tradeoff. For instance, instruction-following benchmarks such as VirtualHome [20] and ALFRED [20, 21] are diverse in the number of scenes, objects, and state changes, but offer a limited low-level physical realism. On the other hand, household rearrangement benchmarks such as Habitat 2.0 HAB [26], TDW Transport [19], and SAPIEN ManiSkill [54, 55] support realistic action execution and accurate physics simulation for rigid bodies, but only include a handful of tasks. Similarly, SoftGym [45] and RFUniverse [56] have the closest simulation features

[illegible]

Table 1: **Comparison of Embodied AI Benchmarks:** BEHAVIOR-1K contains 1,000 diverse activities that are grounded by human needs. It achieves a new level of diversity in scenes, objects, and state changes involved. OMNIGIBSON provides realistic simulation of these 1,000 activities, including some of the most advanced simulation and rendering features such as fluid and deformable bodies. This table is extended from [27].

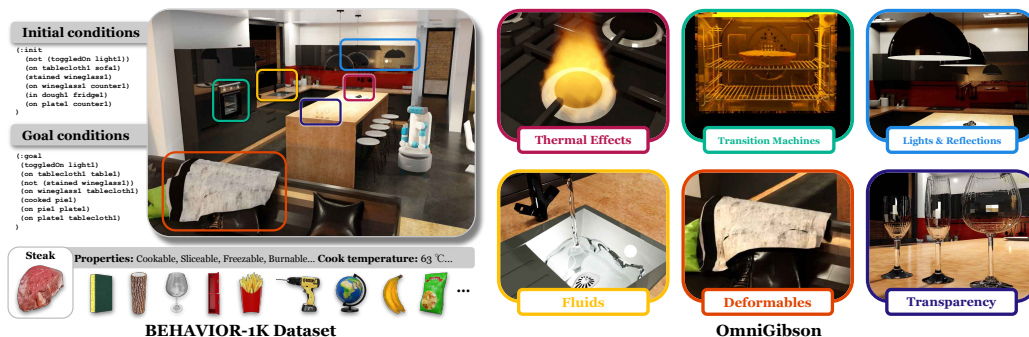


Figure 2: **Elements of BEHAVIOR-1K.** Our benchmark comprises two elements: BEHAVIOR-1K DATASET and OMNIGIBSON. Left: BEHAVIOR-1K DATASET includes 1,000 BDDL activity definitions (top left), 50 realistic and diverse scenes (top right), and 9,000+ objects with properties annotated in the knowledge base (bottom). Right: OMNIGIBSON provides the necessary functionalities to realistically simulate the 1000 activities, including thermal effects such as fire/steam/smoke (top left), fluid dynamics (bottom left), functional machines for transition rules (top center), deformable bodies/cloths (bottom center), realistic lighting and reflections (top right), and transparency rendering (bottom right). Together, they constitute a concrete, realistic instantiation of an everyday activity like `CookingDinner` in simulation.

and hence realism to OMNIGIBSON, but they also lack the task diversity needed to support the development of human-centered general robots.

The most similar benchmark to us is the previous generation BEHAVIOR-100 [27]. BEHAVIOR-100 brought forward several beneficial design choices that we inherit in BEHAVIOR-1K such as the activity sources (ATUS [50]), activity definition logic language, and evaluation metrics. However, it fell short in the diversity and realism necessary to support a human-centered embodied AI benchmark in simulation, dimensions where BEHAVIOR-1K achieves unmatched levels. While BEHAVIOR-100 comprises 100 activities selected by researchers, our BEHAVIOR-1K increases diversity by one order of magnitude, to 1,000 activities, that are grounded in human needs thanks to our unique survey. Furthermore, BEHAVIOR-100 includes only 15 scenes (all houses) and 300+ object categories, while BEHAVIOR-1K increases to 50 scenes (houses, stores, restaurants, offices, etc.) and 1,900+ object categories. In terms of realism, BEHAVIOR-1K extends the simulatable physical states and processes with OMNIGIBSON: fluids, flexible materials, mixing substances, etc. The realism achieved in rendering by OMNIGIBSON for BEHAVIOR-1K is also significantly higher than what was possible in BEHAVIOR-100 and other benchmarks (see Fig. 3).



OMNIGIBSON 3.20 ± 1.23	Habitat 2.0 1.74 ± 1.33	AI2-THOR 1.73 ± 1.37	iGibson 2.0 1.69 ± 1.24	ThreeDWorld 1.65 ± 1.23
---	----------------------------	-------------------------	----------------------------	----------------------------

Figure 3: **Comparison of Visual Realism:** We evaluate OMNIGIBSON’s visual realism against other simulation environments by running a survey with 60 human subjects. We ask them to rank the realism of sampled images from each environment with a score of 5 (most realistic) to 1 (least realistic). We report the mean and standard deviation and show a sampled image from the study. We observe that the participants consider OMNIGIBSON to be significantly more visually realistic than all other environments. See Appendix E.2 for more info.

4 BEHAVIOR-1K DATASET

Once activities have been sourced to reflect human needs, they need to be concretely defined and instantiated the way they would occur in the real world. We build the **BEHAVIOR-1K DATASET**, which includes a knowledge base of crowdsourced activity definitions with relevant objects and object states, and a large-scale repository of high-quality, interactive 3D models.

We crowdsource concrete definitions of activities in the form of **BEHAVIOR Domain Definition Language (BDDL)** [27]. BDDL is based on predicate logic and designed to be accessible for laypeople to describe concrete initial and goal conditions for a given activity. Unlike geometric, image/video, or experience goal specifications [17, 18], BDDL definitions are in terms of objects and object states, allowing annotators to define at an intuitive semantic level. The semantic symbols also capture the fact that multiple physical states might be valid initializations and solutions to an activity. See Listings 1, 2 and 3 in Appendix for example definitions.

The object and object state spaces that activity definitions are built upon are annotated to be ecologically plausible. The object spaces are derived from 5,000 WikiHow articles for the 1,000 activities and mapped to 2,964 leaf-level synsets either from WordNet [57] or custom-designed. Through crowdworkers, students, and GPT-3 [58], we also associate each object with our fully simulatable object states: for example, apple is associated with cooked and sliced, but not toggledOn. Many object-property pairs are augmented with parameters, e.g., “cooked temperature for apples”, taking advantage of OMNIGIBSON’s continuous extended states to make activities especially realistic. Finally, annotators and researchers also create transition rules, e.g., turning tomatoes and salt into sauces, or requiring sandpaper to remove rust. The result is a knowledge base of tens of thousands of elements underlying 1000 ecologically plausible activity definitions. We ensure annotation quality by having five experienced machine learning annotators verify a subset of all types of annotations and receive extremely high approval rates (>96.8%). See Appendix B for more details about the knowledge base.

The diversity of these activity definitions requires diverse object and scene models. On top of the 15 house scenes from BEHAVIOR-100 [27], we acquire 35 fully interactive scenes across diverse scene types, such as gardens, offices, restaurants, and stores, that are essential for everyday activities. This is unprecedented compared to other benchmarks (see Table 1). We also acquire 9,000+ object instances across 1,900+ categories required by the activities, and annotate rich physical (e.g., friction, mass, articulation) and semantic properties (e.g., category) for each object. Representative scene and object models can be seen in Fig. 2. More details of the 3D models can be found in Appendix D.

5 OMNIGIBSON: Instantiating BEHAVIOR-1K with Realistic Simulation

BEHAVIOR-100 is implemented in iGibson 2.0 [59]; however, realistic simulation of the diverse activities in BEHAVIOR-1K is beyond the capability of iGibson 2.0. We present a novel simulation environment, OMNIGIBSON, that provides the necessary functionalities to support and instantiate BEHAVIOR-1K. OMNIGIBSON is built on top of Nvidia Omniverse and PhysX 5, providing the simulation of not only rigid bodies, but also deformable objects, fluids, and flexible materials (see

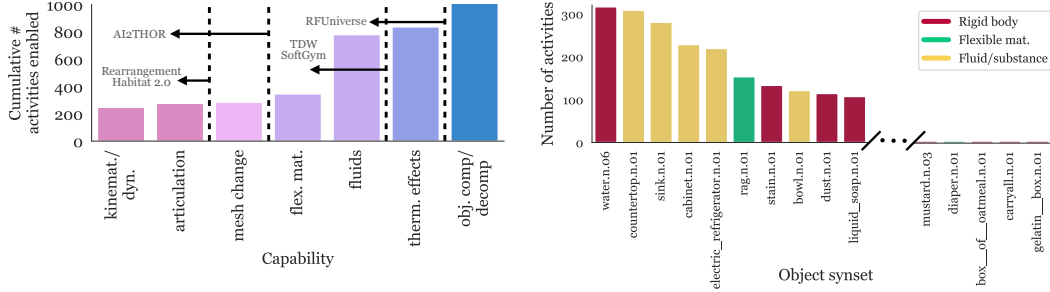


Figure 4: **Objects and States in Activity Definitions:** Left: the number of activities unlocked by each simulation capability that OMNIGIBSON has. None of the other simulation environments are sufficient to fully support BEHAVIOR-1K, e.g. Habitat 2.0 can support only 23% of the activities. Right: the number of activities that require each object synset (category). Several top-10 object synsets are fluids and flexible materials, necessitating the development of OMNIGIBSON. As we expect, the object synsets also follow a long-tail distribution: most objects are involved in only a few activities.

Fig. 4), while generating highly realistic ray-traced or path-traced virtual images (see Fig. 3). These features significantly boost the realism of BEHAVIOR-1K compared to other benchmarks.

Similar to BEHAVIOR-100, OMNIGIBSON also simulates additional, non-kinematic extended object states (e.g., temperature, soaked level) based on heuristics (e.g., temperature increases when being next to a heat source that is toggled on). OMNIGIBSON also implements the functionalities to generate infinite valid physical configurations that satisfy the activities’ initial conditions as logical predicates (e.g. food is frozen), and to evaluate their goal conditions (e.g. food is cooked and onTop of a plate, the cloth is folded) based on the object’s physical states (pose and joint configuration) and extended states. OMNIGIBSON natively supports randomization during scene initialization and can sample amongst object models and their poses/states. The full details of extended object states and logical predicates that OMNIGIBSON supports can be found in Appendix E.1.

Many everyday tasks are difficult to simulate because they require modeling complex physical processes, such as folding a towel or pouring a glass of water. OMNIGIBSON unlocks them by supporting realistic simulation of fluids, deformable bodies, and cloths (see Fig. 2). Indeed, without these features, over half of BEHAVIOR-1K activities would not be simulatable, highlighting how crucial these features are for capturing everyday activities. OMNIGIBSON also captures multiple physical processes that are not natively simulatable by Omniverse, such as baking pies or pureeing vegetables. Aside from the extended states mentioned above, we also design a modular *Transition Machine*, which specifies custom transitions between groups of objects when specified conditions are met. For example, a dough placed inside an oven that reaches a certain temperature threshold will turn into a pie. This further expands OMNIGIBSON’s capacity to simulate complex, realistic activities that would otherwise be intractable to fully simulate physically.

6 Experiments: Evaluating Embodied AI Solutions in BEHAVIOR-1K

In our experiments, we aim to answer three questions: How do existing vision-based robot learning algorithms perform in BEHAVIOR-1K, and what assumptions have to be made to improve their success? What elements of the activities are the most problematic for current AI? What are the main sources of the sim-real gap in BEHAVIOR-1K/OMNIGIBSON? Our goal is to indicate promising research directions to improve AI’s performance in BEHAVIOR-1K activities in simulation and, ultimately, in the real world.

6.1 Evaluating BEHAVIOR-1K Solutions in OMNIGIBSON

Experimental Setups. We selected three paradigmatic activities for our experiments: CollectTrash, where the agent gathers empty bottles and cups, and throws them into a trash bin (rigid body manipulation); StoreDecoration, where the agent stores items into a drawer (articulated object manipulation); and CleanTable, where the agent wipes a dirty table with a soaked piece of cloth

Method	Policy Features		Task success rate		
	Primitives	History	StoreDecoration	CollectTrash	CleanTable
RL-VMC	✗	✗	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
RL-Prim.	✓	✗	0.48 ± 0.06	0.42 ± 0.02	0.77 ± 0.08
RL-Prim.Hist.	✓	✓	0.55 ± 0.05	0.63 ± 0.03	0.88 ± 0.02

Table 2: Task success rates across three baseline methods. RL-VMC with end-to-end visuomotor control completely fails to solve any of the activities, whereas RL-Prim. and RL-Prim.Hist. with action primitives are able achieve decent performance. Memory of observations helps in longer horizon activities (e.g. CollectTrash).

(manipulation of flexible materials and fluids). We evaluate three different baselines based on state-of-the-art reinforcement learning algorithms (RL) [60]:

- RL-VMC, a visuomotor control (from image to low-level joint commands) RL solution based on Soft Actor-Critic (SAC) [48];
- RL-Prim., a RL solution based on PPO [47] that leverages a set of action primitives based on a sampling-based motion planner [61, 62, 49] (pick, place, push, navigate, dip and wipe). The policy outputs a discrete selection of a primitive applied on an object;
- RL-Prim.Hist., a variant of RL-Prim. that takes in the history observations (3 steps) as additional inputs to help disentangle similar-looking states.

All agents are trained with a sparse task success reward without any reward engineering. Following the metrics proposed in BEHAVIOR-100 [27], we report the success rate and efficiency metrics (distance traveled, time invested, and disarrangement caused) in Table 2 and 3, and the success score Q in Table A.13 in Appendix.

Grasping is a challenging research topic on its own. To facilitate our experiments, we adopt an assistive pick primitive that creates a rigid connection between the object and the gripper if grasping is requested when all fingers are in contact with the object, a stricter form of *StickyMitten* used in prior works [26, 63, 64]. Furthermore, to accelerate training, the action primitives check only the feasibility (e.g., reachability, collisions) of the final configuration, e.g. the grasping pose for pick or the desired location for navigate. If kinematically feasible, the action primitives will directly set the robot state to the final configuration, and continue to simulate from then on. We include an ablation analysis of the effect of these assumptions and simplifications in our evaluation (see Table 4). Further details about our training and evaluation setup can be found in Appendix F.

Results: Task Completion. Table 2 contains task success rates across our baseline methods. The extreme long-horizon in our activities causes the visuomotor control (RL-VMC) policy to fail in all three activities, potentially due to problems such as credit assignment [65], deep exploration [66, 67], and vanishing gradients [68] as reported by prior works. Our baselines with time-extended action primitives (RL-Prim. and RL-Prim.Hist.) obtain better success, achieving over 40% success rates across all three activities. We observe that longer-horizon activities are more challenging: while CleanTable can be accomplished by executing the optimal sequence of 6 primitive steps, CollectTrash requires at least 16. This supports the idea that some form of action-space abstraction must be necessary to solve long-horizon activities of BEHAVIOR-1K, as others reported [27, 26, 69]. When analyzing the role of memory, we observe a sizable performance gain from RL-Prim. to RL-Prim.Hist., especially in long-horizon activities with aliased observations such as CollectTrash. In this task, when the robot is looking at the trash bin, it needs additional information to know what location has been cleaned already in order to proceed to other locations. Our results indicate that memory will play a critical role for embodied AI in long-horizon BEHAVIOR-1K activities.

Results: Efficiency. In addition to success, efficiency is also critical in the evaluation of embodied AI: a successful policy in simulation may be infeasible in the real world if it takes a long time or wastes too much energy. In Table 3, we report the results with three efficiency metrics proposed by Srivastava et al. [27]. We observe that the use of memory (RL-Prim.Hist.) improves efficiency across all metrics: distance navigated (Dist. Nav.), simulated time (Sim. Time), and kinematic object disarrangement (Kin. Dis.), i.e., amount of object displacement due to robot motion.

Method	Metrics in CollectTrash		
	Dist. Nav. [m]	Sim. Time [s]	Kin. Dis. [m]
RL-VMC	27.58 $\pm$ 5.95	16.67 $\pm$ 0.00	0.00 $\pm$ 0.00
RL-Prim.	17.98 $\pm$ 2.35	13.95 $\pm$ 5.14	12.34 $\pm$ 5.01
RL-Prim.Hist.	15.33 $\pm$ 2.70	12.48 $\pm$ 3.68	10.82 $\pm$ 3.90

Table 3: Efficiency metrics across three baseline methods. RL-VMC has low spatial and temporal efficiency because it fails to learn, whereas history information helps remove redundant actions and improve efficiency.

Phys. Realism		Task success rate		
Grasping	Full Motion	StoreDecoration	CollectTrash	CleanTable
✓	✓	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0
✗	✓	0.46 $\pm$ 0.04	0.36 $\pm$ 0.08	0.73 $\pm$ 0.03
✗	✗	0.48 $\pm$ 0.06	0.42 $\pm$ 0.02	0.77 $\pm$ 0.08

Table 4: Ablation study of RL-Prim. on the impact of removing the simplifying assumptions of grasping and motion execution during evaluation. We observe a large drop in performance when enabling fully physics-based grasping, but not when enabling full trajectory motion execution.



Figure 5: **Characterizing the Sim-Real Gap:** Left: a side-by-side comparison of the simulated and the real scene, including virtual and real images obtained by the robot. While the high-resolution images are extremely similar, the mismatch in wooden texture and camera properties causes a sizable gap in the visual input to the agent. Right: source of failure in Simulation (S, left) and in Real-world with a Trained Policy in OMNIGIBSON (R-TP, middle) and an Optimal Policy (R-OP, right) due to actuation (solid color) or perception (striped). In simulation, without full simulation of grasping (see Sec. 6.1), policy failures (i.e., selecting the wrong action primitive) dominate. On the real robot, grasping is one of the main causes of error, as well as perception issues (policy errors with the trained visual policy, object detection errors with the optimal policy).

We also evaluate to what extent the simplifications we introduce in physics and actuation (grasping, motion execution) during training impact the performance of RL-Prim. during evaluation when these simplifications are removed. We report the results in Table 4. We observe a radical performance drop after enabling fully physics-based grasping during evaluation. Grasping is thus a critical component of any embodied AI task, and researchers should be careful when simplifying its execution during training. While OMNIGIBSON supports fully physics-based grasping, designing a pick action primitive for arbitrary objects that leverages fully physics-based grasping is by itself an open research problem that we leave for future work. In contrast, there is much less performance drop after enabling full trajectory motion execution during evaluation. This result supports our hypothesis that it is reasonable to accelerate the training process by assuming that motion planning is likely to provide viable paths in free space during evaluation.

6.2 Evaluating BEHAVIOR-1K Solutions on a Real Robot

We performed a series of experiments with a real robot to answer the question: *what are the main sources of discrepancy between our realistic simulation and the real world?* To that end, we used a real-world counterpart of the simulated scene of a mockup apartment for the CollectTrash activity. We scanned the apartment and converted it into a virtual, interactive scene. We use a real bi-manual mobile manipulator Tiago, and leverage the RGB-D images from its onboard sensors and a YOLOv3 object detector [70, 71] to localize the objects in 3D space for manipulation. For navigation, the robot localizes with a particle filter [72] based on two LiDAR sensors and a map of the apartment. The action primitives are implemented with the same sampling-based motion planning algorithm as in simulation [62, 49] with additional tuning. We evaluate two strategies for selecting action primitives in the real world: an optimal policy based on human input, and a vision-based policy (RL-Prim.) trained in OMNIGIBSON. To facilitate sim-to-real transfer, during training we additionally applied image-based data augmentation to the observations based on a prior work [73] (see Appendix G for further details). With the optimal policy, we evaluate the gap in actuation between the simulated and the real robot; with the learned policy, we also evaluate the gap in visual perception. We achieve different success rates in simulation (50 runs, $\sim$ 40% success) and in the real world with optimal (27 runs, $\sim$ 22%) and trained policies (26 runs, 0%), hinting at a sim-real gap that we analyze below.

The failure cases are depicted in Fig. 5 (right). We observe that the majority of failures in simulation are due to the visual policy (perception), while others are caused by stochasticity in the place primitive

and the sampling-based motion planner. The reason why none of the failures are due to grasping is that, in simulation, we evaluate with the assistive pick primitive. Grasping is fairly difficult in the real world, contributing to around 40% of the failures for both the trained and the optimal policy. For the learned policy, 44% of the errors come from the visual policy selecting the wrong action primitive due to the differences between the simulated and the real images. The visual discrepancy results from unmodeled effects such as the real camera’s poor dynamic range (see Fig. 5, left and middle) and imperfect object modeling (e.g. the exact wooden texture and the surface reflectivity of the tables), which can be alleviated by more targeted domain randomization. Interestingly, several manipulation failures on the real robot are caused by unfavorable robot base placement resulting from navigation inaccuracies in the previous timestep. This compounding source of error is not present in simulation because we assume perfect localization and execution.

We believe this analysis provides relevant information about the main sources and severity of the sim-real gap in BEHAVIOR-1K in OMNIGIBSON, and provide insights for future research avenues. Our plan is to use some of these insights to create novel sim-to-real solutions that make progress on BEHAVIOR-1K.

7 Discussion and Limitations

We have presented BEHAVIOR-1K, a benchmark for embodied AI and robotics research with realistic simulation of 1000 diverse activities grounded in human needs. BEHAVIOR-1K comprises two elements: BEHAVIOR-1K DATASET, a semantic knowledge base of everyday activities, and a large-scale 3D model library; OMNIGIBSON, a simulation environment that provides realistic rendering and physics for rigid/deformable objects, flexible materials and fluids. In our evaluation, we observed that BEHAVIOR-1K is an extremely challenging benchmark: solving these 1,000 activities autonomously is beyond the capability of current state-of-the-art AI algorithms. We studied and attempted to solve a handful of the activities with action primitives in order to gain insights into the most challenging components, providing a starting point for other researchers to work on our benchmark. Similarly, we explored the sources of the sim-real gap by creating a digital twin of a real-world mock apartment, and by performing rigorous evaluation and analysis of policies in both simulation and the real world with a simulated and real mobile manipulator.

Limitations: We inherit several limitations from our underlying physics and rendering engine, Nvidia’s Omniverse. In OMNIGIBSON, we trade off rendering speed for visual realism (ray-traced), reaching around 60 fps for a house scene of around 60 objects (v.s. around 100 fps in iGibson 2.0 [59]). We are actively working on performance optimization. Another limitation is that we only include activities that do not require interactions with humans. Realistic simulation of humans (behavior, motion, appearance) is extremely challenging and an open research area. We plan to include simulated humans as technology matures. Finally, there is still room for improvement in OMNIGIBSON to further facilitate sim2real transfer, such as incorporating noise models of perception and actuation.

Acknowledgments

This work was done in part when Chengshu Li, Josiah Wong, and Michael Lingelbach were interns at Nvidia Research. The work is in part supported by the Stanford Institute for Human-Centered AI (HAI), the Toyota Research Institute (TRI), NSF CCRI #2120095, NSF RI #2211258, NSF NRI #2024247, ONR MURI N00014-22-1-2740, ONR MURI N00014-21-1-2801, Amazon, Bosch, Salesforce, and Samsung. Ruohan Zhang is partially supported by the Wu Tsai Human Performance Alliance Fellowship. Sanjana Srivastava is partially supported by the National Science Foundation Graduate Research Fellowship Program (NSF GRFP).

References

- [1] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. Imagenet: A large-scale hierarchical image database. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.

- [2] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft coco: Common objects in context. In *European Conference on Computer Vision*, pages 740–755. Springer, 2014.
- [3] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman. The pascal visual object classes (voc) challenge. *International Journal of Computer Vision*, 88(2):303–338, 2010.
- [4] R. Krishna, Y. Zhu, O. Groth, J. Johnson, K. Hata, J. Kravitz, S. Chen, Y. Kalantidis, L.-J. Li, D. A. Shamma, et al. Visual genome: Connecting language and vision using crowdsourced dense image annotations. *International Journal of Computer Vision*, 123(1):32–73, 2017.
- [5] A. Geiger, P. Lenz, and R. Urtasun. Are we ready for autonomous driving? the kitti vision benchmark suite. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 3354–3361. IEEE, 2012.
- [6] R. Goyal, S. Ebrahimi Kahou, V. Michalski, J. Materzynska, S. Westphal, H. Kim, V. Haenel, I. Fruend, P. Yianilos, M. Mueller-Freitag, et al. The "something something" video database for learning and evaluating visual common sense. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 5842–5850, 2017.
- [7] G. A. Sigurdsson, A. Gupta, C. Schmid, A. Farhadi, and K. Alahari. Actor and observer: Joint modeling of first and third-person videos. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 7396–7404, 2018.
- [8] Y. Xiang, T. Schmidt, V. Narayanan, and D. Fox. Posecnn: A convolutional neural network for 6d object pose estimation in cluttered scenes. *arXiv preprint arXiv:1711.00199*, 2017.
- [9] R. Martín-Martín, M. Patel, H. Rezatofighi, A. Shenoi, J. Gwak, E. Frankel, A. Sadeghian, and S. Savarese. Jrdp: A dataset and benchmark of egocentric robot visual perception of humans in built environments. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021.
- [10] F. Caba Heilbron, V. Escorcia, B. Ghanem, and J. Carlos Nibbles. Activitynet: A large-scale video benchmark for human activity understanding. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 961–970, 2015.
- [11] D. Gurari, Q. Li, A. J. Stangl, A. Guo, C. Lin, K. Grauman, J. Luo, and J. P. Bigham. Vizwiz grand challenge: Answering visual questions from blind people. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3608–3617, 2018.
- [12] M. A. Marcinkiewicz. Building a large annotated corpus of english: The penn treebank. *Using Large Corpora*, page 273, 1994.
- [13] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding. *arXiv preprint arXiv:1804.07461*, 2018.
- [14] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang. Squad: 100,000+ questions for machine comprehension of text. *arXiv preprint arXiv:1606.05250*, 2016.
- [15] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Y. Ng, and C. Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, pages 1631–1642, 2013.
- [16] S. Antol, A. Agrawal, J. Lu, M. Mitchell, D. Batra, C. L. Zitnick, and D. Parikh. Vqa: Visual question answering. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2425–2433, 2015.
- [17] D. Batra, A. X. Chang, S. Chernova, A. J. Davison, J. Deng, V. Koltun, S. Levine, J. Malik, I. Mordatch, R. Mottaghi, M. Savva, and H. Su. Rearrangement: A challenge for embodied ai. *arXiv preprint arXiv:2011.01975*, 2020.

- [18] L. Weihs, M. Deitke, A. Kembhavi, and R. Mottaghi. Visual room rearrangement. *arXiv preprint arXiv:2103.16544*, 2021.
- [19] C. Gan, S. Zhou, J. Schwartz, S. Alter, A. Bhandwaldar, D. Gutfreund, D. L. Yamins, J. J. DiCarlo, J. McDermott, A. Torralba, et al. The threedworld transport challenge: A visually guided task-and-motion planning benchmark for physically realistic embodied ai. *arXiv preprint arXiv:2103.14025*, 2021.
- [20] X. Puig et al. Virtualhome: Simulating household activities via programs. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018.
- [21] M. Shridhar, J. Thomason, D. Gordon, Y. Bisk, W. Han, R. Mottaghi, L. Zettlemoyer, and D. Fox. Alfred: A benchmark for interpreting grounded instructions for everyday tasks. In *Proceedings of the IEEE/CVF conference on Computer Vision and Pattern Recognition*, pages 10740–10749, 2020.
- [22] F. Xia, W. B. Shen, C. Li, P. Kasimbeg, M. E. Tchapmi, A. Toshev, R. Martín-Martín, and S. Savarese. Interactive gibbon benchmark: A benchmark for interactive navigation in cluttered environments. *IEEE Robotics and Automation Letters*, 5(2):713–720, 2020.
- [23] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, and S. Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Conference on Robot Learning*, pages 1094–1100. PMLR, 2020.
- [24] S. James, Z. Ma, D. Rovick Arrojo, and A. J. Davison. Rlbench: The robot learning benchmark & learning environment. *IEEE Robotics and Automation Letters*, 2020.
- [25] M. Savva, A. Kadian, O. Maksymets, Y. Zhao, E. Wijmans, B. Jain, J. Straub, J. Liu, V. Koltun, J. Malik, et al. Habitat: A platform for embodied ai research. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9339–9347, 2019.
- [26] A. Szot, A. Clegg, E. Undersander, E. Wijmans, Y. Zhao, J. Turner, N. Maestre, M. Mukadam, D. S. Chaplot, O. Maksymets, et al. Habitat 2.0: Training home assistants to rearrange their habitat. In *Advances in Neural Information Processing Systems*, volume 34, 2021.
- [27] S. Srivastava, C. Li, M. Lingelbach, R. Martín-Martín, F. Xia, K. E. Vainio, Z. Lian, C. Gokmen, S. Buch, K. Liu, et al. Behavior: Benchmark for everyday household activities in virtual, interactive, and ecological environments. In *Conference on Robot Learning*, pages 477–490. PMLR, 2022.
- [28] Y. Zhu, J. Wong, A. Mandlekar, and R. Martín-Martín. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020.
- [29] Y. Tassa, Y. Doron, A. Muldal, T. Erez, Y. Li, D. d. L. Casas, D. Budden, A. Abdolmaleki, J. Merel, A. Lefrancq, et al. Deepmind control suite. *arXiv preprint arXiv:1801.00690*, 2018.
- [30] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.
- [31] M. L. Littman, I. Ajunwa, G. Berger, C. Boutilier, M. Currie, F. Doshi-Velez, G. Hadfield, M. C. Horowitz, C. Isbell, H. Kitano, et al. Gathering strength, gathering storms: The one hundred year study on artificial intelligence (AI100) 2021 study panel report. Technical report, Stanford University, 2021.
- [32] M. O. Riedl. Human-centered artificial intelligence and machine learning. *Human Behavior and Emerging Technologies*, 1(1):33–36, 2019.
- [33] W. Xu. Toward human-centered ai: a perspective from human-computer interaction. *Interactions*, 26(4):42–46, 2019.

- [34] B. Shneiderman. Bridging the gap between ethics and practice: guidelines for reliable, safe, and trustworthy human-centered ai systems. *ACM Transactions on Interactive Intelligent Systems (TiiS)*, 10(4):1–31, 2020.
- [35] H. Kitano, M. Asada, Y. Kuniyoshi, I. Noda, E. Osawa, and H. Matsubara. Robocup: A challenge problem for ai. *AI magazine*, 18(1):73–73, 1997.
- [36] T. Wisspeintner, T. Van Der Zant, L. Iocchi, and S. Schiffer. Robocup@home: Scientific competition and benchmarking for domestic service robots. *Interaction Studies*, 10(3):392–426, 2009.
- [37] L. Iocchi, D. Holz, J. Ruiz-del Solar, K. Sugiura, and T. Van Der Zant. Robocup@ home: Analysis and results of evolving competitions for domestic and service robots. *Artificial Intelligence*, 229:258–281, 2015.
- [38] M. Buehler, K. Iagnemma, and S. Singh. *The DARPA Urban Challenge: Autonomous Vehicles in City Traffic*, volume 56. springer, 2009.
- [39] E. Krotkov, D. Hackett, L. Jackel, M. Perschbacher, J. Pippine, J. Strauss, G. Pratt, and C. Orłowski. The darpa robotics challenge finals: Results and perspectives. *Journal of Field Robotics*, 34(2):229–240, 2017.
- [40] N. Correll, K. E. Bekris, D. Berenson, O. Brock, A. Causo, K. Hauser, K. Okada, A. Rodriguez, J. M. Romano, and P. R. Wurman. Analysis and observations from the first amazon picking challenge. *IEEE Transactions on Automation Science and Engineering*, 15(1):172–188, 2016.
- [41] C. Eppner, S. Höfer, R. Jonschkowski, R. Martín-Martín, A. Sieverling, V. Wall, and O. Brock. Lessons from the amazon picking challenge: four aspects of building robotic systems. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence*, pages 4831–4835, 2017.
- [42] M. A. Roa, M. Dogar, C. Vivas, A. Morales, N. Correll, M. Gerner, J. Rosell, S. Foix, R. Memmesheimer, F. Ferro, et al. Mobile manipulation hackathon: Moving into real world applications. *IEEE Robotics & Automation Magazine*, pages 2–14, 2021.
- [43] E. Heiden, M. Macklin, Y. Narang, D. Fox, A. Garg, and F. Ramos. Disect: A differentiable simulation engine for autonomous robotic cutting. *arXiv preprint arXiv:2105.12244*, 2021.
- [44] Y. Urakami, A. Hodgkinson, C. Carlin, R. Leu, L. Rigazio, and P. Abbeel. Doorgym: A scalable door opening environment and baseline agent. *arXiv preprint arXiv:1908.01887*, 2019.
- [45] X. Lin, Y. Wang, J. Olkin, and D. Held. Softgym: Benchmarking deep reinforcement learning for deformable object manipulation. In *Conference on Robot Learning*, 2020.
- [46] Nvidia, Corp. Physx. <https://developer.nvidia.com/physx-sdk>, 2022. Accessed: 2022-06-10.
- [47] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [48] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, pages 1861–1870. PMLR, 2018.
- [49] M. Jordan and A. Perez. Optimal bidirectional rapidly-exploring random trees. Technical Report MIT-CSAIL-TR-2013-021, Computer Science and Artificial Intelligence Laboratory, Massachusetts Institute of Technology, Cambridge, MA, 2013.
- [50] U.S. Bureau of Labor Statistics. American Time Use Survey. <https://www.bls.gov/tus/>, 2019.

- [51] European Commission. Harmonised european time use surveys. <https://ec.europa.eu/eurostat/web/time-use-surveys>, 2010.
- [52] J. Gershuny, M. Vega-Rapun, and J. Lamote. Multinational time use study. <https://www.timeuse.org/mtus>, 2020.
- [53] wikiHow, Inc. wikihow. <https://www.wikihow.com>, 2021. Accessed: 2021-06-16.
- [54] F. Xiang, Y. Qin, K. Mo, Y. Xia, H. Zhu, F. Liu, M. Liu, H. Jiang, Y. Yuan, H. Wang, et al. SAPIEN: A simulated part-based interactive environment. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11097–11107, 2020.
- [55] T. Mu, Z. Ling, F. Xiang, D. Yang, X. Li, S. Tao, Z. Huang, Z. Jia, and H. Su. Maniskill: Generalizable manipulation skill benchmark with large-scale demonstrations. *arXiv preprint arXiv:2107.14483*, 2021.
- [56] H. Fu, W. Xu, H. Xue, H. Yang, R. Ye, Y. Huang, Z. Xue, Y. Wang, and C. Lu. Rfuniverse: A physics-based action-centric interactive environment for everyday household tasks. *arXiv preprint arXiv:2202.00199*, 2022.
- [57] G. A. Miller. Wordnet: a lexical database for english. *Communications of the ACM*, 38(11): 39–41, 1995.
- [58] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.
- [59] C. Li, F. Xia, R. Martín-Martín, M. Lingelbach, S. Srivastava, B. Shen, K. E. Vainio, C. Gokmen, G. Dharan, T. Jain, A. Kurenkov, K. Liu, H. Gweon, J. Wu, L. Fei-Fei, and S. Savarese. igibson 2.0: Object-centric simulation for robot learning of everyday household tasks. In *Annual Conference on Robot Learning*, 2021.
- [60] A. G. Barto and S. Mahadevan. Recent advances in hierarchical reinforcement learning. *Discrete Event Dynamic Systems*, 13(1):41–77, 2003.
- [61] S. M. LaValle. *Planning Algorithms*. Cambridge University Press, 2006.
- [62] J. J. Kuffner and S. M. LaValle. Rrt-connect: An efficient approach to single-query path planning. In *Proceedings IEEE International Conference on Robotics and Automation*, volume 2, pages 995–1001. IEEE, 2000.
- [63] K. Ehsani, W. Han, A. Herrasti, E. VanderBilt, L. Weihs, E. Kolve, A. Kembhavi, and R. Motlaghi. Manipulathor: A framework for visual object manipulation. In *Proceedings of the IEEE/CVF conference on Computer Vision and Pattern Recognition*, pages 4497–4506, 2021.
- [64] C. Li, F. Xia, R. Martín-Martín, and S. Savarese. Hrl4in: Hierarchical reinforcement learning for interactive navigation with mobile manipulators. In *Conference on Robot Learning*, pages 603–616. PMLR, 2020.
- [65] V. Alipov, R. Simmons-Edler, N. Putintsev, P. Kalinin, and D. Vetrov. Towards practical credit assignment for deep reinforcement learning. *arXiv preprint arXiv:2106.04499*, 2021.
- [66] T. Yang, H. Tang, C. Bai, J. Liu, J. Hao, Z. Meng, and P. Liu. Exploration in deep reinforcement learning: a comprehensive survey. *arXiv preprint arXiv:2109.06668*, 2021.
- [67] I. Osband, B. V. Roy, D. J. Russo, and Z. Wen. Deep exploration via randomized value functions. *Journal of Machine Learning Research*, 20(124):1–62, 2019.
- [68] S. Hochreiter. The vanishing gradient problem during learning recurrent neural nets and problem solutions. *Int. J. Uncertain. Fuzziness Knowl.-Based Syst.*, 6(2):107–116, 1998.

- [69] F. Xia, C. Li, R. Martín-Martín, O. Litany, A. Toshev, and S. Savarese. ReLMoGen: Leveraging motion generation in reinforcement learning for mobile manipulation. In *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020.
- [70] J. Redmon and A. Farhadi. Yolov3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.
- [71] M. Bjelonic. YOLO ROS: Real-time object detection for ROS. https://github.com/leggedrobotics/darknet_ros, 2016–2018.
- [72] S. Thrun, W. Burgard, and D. Fox. *Probabilistic Robotics*. MIT Press, 2005.
- [73] L. Fan, G. Wang, D.-A. Huang, Z. Yu, L. Fei-Fei, Y. Zhu, and A. Anandkumar. Secant: Self-expert cloning for zero-shot generalization of visual policies. *arXiv preprint arXiv:2106.09678*, 2021.
- [74] M. Koupaee and W. Y. Wang. Wikihow: A large scale text summarization dataset. *arXiv preprint arXiv:1810.09305*, 2018.
- [75] R. Likert. A technique for the measurement of attitudes. *Archives of psychology*, 1932.
- [76] D. C. Montgomery. *Design and analysis of experiments*. John Wiley & Sons, Inc., Hoboken, NJ, eighth edition, 2013.
- [77] G. Paolacci, J. Chandler, and P. G. Ipeirotis. Running experiments on amazon mechanical turk. *Judgment and Decision making*, 5(5):411–419, 2010.
- [78] P. R. Center. Research in the crowdsourcing age, a case study. Technical report, Washington, D.C., July 2016. URL <https://www.pewresearch.org/internet/2016/07/11/research-in-the-crowdsourcing-age-a-case-study/>.
- [79] A. Akbik, D. Blythe, and R. Vollgraf. Contextual string embeddings for sequence labeling. In *COLING 2018, 27th International Conference on Computational Linguistics*, pages 1638–1649, 2018.
- [80] M. Fox and D. Long. PDDL2.1: An extension to PDDL for expressing temporal planning domains. *Journal of Artificial Intelligence Research*, 20:61–124, dec 2003. doi:10.1613/jair.1129. URL <https://doi.org/10.1613%2Fjair.1129>.
- [81] J. Wichlacz, Álvaro Torralba, and J. Hoffmann. Construction-planning models in minecraft. In *Proceedings of the 2nd ICAPS Workshop on Hierarchical Planning (HPlan 2019)*, pages 1–5, 2019.
- [82] TurboSquid, Inc. Turbosquid. <https://www.turbosquid.com/>, 2022. Accessed: 2022-06-24.
- [83] Niantic, Inc. Scaniverse. <https://scaniverse.com>, 2022. Accessed: 2022-06-24.
- [84] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, A. Y. Ng, et al. Ros: an open-source robot operating system. In *ICRA workshop on open source software*, volume 3, page 5. Kobe, Japan, 2009.
- [85] D. Baboolall, D. Pinder, and S. Stewart. How automation could affect employment for women in the united kingdom and minorities in the united states. *McKinsey Digital*, 2019. URL tinyurl.com/h883dm9n.

A Survey

In this section, we provide further information about the survey presented in Sec. 2 that guided our benchmark development. We’ll discuss how we selected activities that are part of the survey, the survey design and execution, and the demographic information about the survey participants.

A.1 Activity Sources

We source activities from a combination of *time-use surveys* and WikiHow [74]. Time-use surveys are studies that inquire about the daily time use of a population [50], making them a good proxy for daily requirements of embodied intelligence. We combine information from three time-use surveys—American [50], Harmonized European [51], and Multinational [52] Time Use Surveys—and obtain an initial set of 540 activities. The time-use surveys focus on activities that happen with enough frequency and require a significant amount of time. However, there are other activities that are essential for everyday human life, but not reflected in the time-use surveys. WikiHow articles include activities that are important for humans where they seek guidance, even if they are not as frequent as the ones included in the time-use surveys. This indicates a great potential to source additional useful and relevant daily activities. We complement the activities from the time use surveys with WikiHow article titles, of which there are 180,000+. These constitute a raw set of activities that are filtered down by feasibility, as explained in the next paragraph.

Activity filtering for feasibility: Simulation constrains which activities collected from time-use surveys and WikiHow can actually be used in BEHAVIOR-1K. We used the following criteria based on OMNIGIBSON and BDDL constraints to filter the activity pool prior to the survey:

Filtering principle	Example activity filtered out
Activity requires physics or chemistry not supported in simulation	SteamingClothes, MakingSoap
Activity involves creating or consuming media	ReadingABook
Activity requires more than a day in real time	DryingSeedsOvernight
Activity requires non-visual perceptual modalities	SweeteningFood
Activity requires geometric configurations too fine-grained for BDDL	SettingUpANativityScene
Activity is predicated on branded items	SprayingWindex
Activity involves other people or live animals	AskingForARaise

After filtering and eliminating duplicates, and including the activities from [27], 2,090 activities remain to be surveyed.

A.2 Survey Design

Our survey is structured as follows:

- **Demographic questions** requesting information about the number of people in the household, occupation, general location, and relationship between household work, automation, and livelihood (see Fig. A.1 for results).
- **Activity survey questions** requesting the value of automation of a batch of activities to the respondent. Specifically, this section comprises:
 - 50 questions, one question per activity
 - Question text: *On a scale of 1 (left) to 10 (right), rate how much you want a robot to do this activity for you.*
 - Each response uses an independent Likert score [75] on a scale from 1 (less beneficial) to 10 (most beneficial)

We piloted alternatives for the survey about two design decisions: 1) question wording, and 2) question format. With respect to the wording for the question, we piloted three alternatives to specify the *agent*: *robot*, *assistant*, and *automation*. Our goal was to study any possible (mis)conceptions about robots that might bias the study. By way of 30 pairwise T-tests and 10 ANOVAS [76], we did not find any statistically significant difference between the results obtained using these three wordings. With respect to the format of the question, we piloted two alternative formats: 10-point Likert, and three-element best-worst scaling. By way of Kendall’s tau [76], there was a strong correlation between the rankings determined by Likert scores and standard metrics of best-worst scaling. We, therefore, conclude that either method will give us a similar ranking, and select the more resource-efficient option (Likert).

Survey collection: We deployed our survey on Amazon Mechanical Turk [77] and collected 50 unique responses per activity. There were a total of 1,461 different respondents and their average

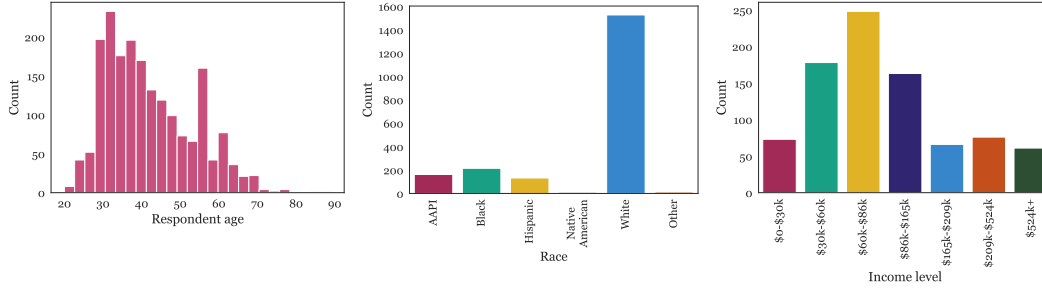


Figure A.1: **Demographic distribution of the participants in our survey.** Respondents appear to have similar demographics to the wider U.S. Mechanical Turk worker population [78]. Not pictured: respondent gender (43.41% women, 55.50% men, 0.83% non-binary, 0.26% other), disability status (92.56% no, 5.74% yes, 1.70% prefer not to answer).

scores ranged from 1.9 to 9.3, showing high diversity. The average score was 5.16. To ensure response quality, we repeated four questions in every survey as an attention check. If responses to a pair of repeats differed by more than two points, we considered it failed. Two or more failures led to rejection. We also rejected survey responses that had no significant variance across their responses to 50 different activities.

A.3 Demographic Information

Fig. A.1 depicts the results of our demographic questions on the participants of the survey. We observe that most adult age groups have good representation, particularly pre-retirement age groups, and are concentrated in the 30-40 group. Racially, the respondents are around 75% white, a larger proportion than the U.S. population but similar to the Mechanical Turk proportion [78]. Other races appear to have proportions similar to but not the same as their presence on both Mechanical Turk and in the general population; this may be due to their overall small numbers in all three [78]. There is some Native American representation, though not statistically significant. Income-wise, respondents tend to be in the \$30,000-\$150,000 range, particularly in the lower half.

The participants’ gender is distributed by 43.41% women, 55.50% men, 0.83% non-binary, 0.26% other. Gender representation is not as even as the U.S. population but more so than the Mechanical Turk worker population [78]. There is a small representation of non-binary individuals. When it comes to disability status, our participants reported 92.56% no-disability, 5.74% disability, and 1.70% prefer not to answer. Thus, our survey contained some but limited representation of people with disabilities. This group, together with the elderly, are groups commonly assumed to potentially benefit from robotic efforts; they could be subject to future targeted surveys.

B Activity Annotation

Obtaining the BDDL definitions for 1,000 activities requires multiple preparatory steps to gather the knowledge needed. This knowledge also becomes part of the BEHAVIOR-1K DATASET. The pipeline is outlined in Fig. A.2. We now detail each annotation other than the survey. Furthermore, quality assessment statistics for these annotations are found in Sec. B.1. We see that experienced annotators find our resulting BEHAVIOR-1K knowledge base highly accurate.

Obtaining list of objects per activity (WikiHow articles, noun phrase extraction, and manual object filtering): Our activity definition requires first defining a set of objects and properties that the annotators could use to describe an activity. We would like these domains to be natural and ecological, containing all relevant objects that humans may consider necessary for a task. To obtain such an ecologically plausible object space per activity, we parse WikiHow articles. Since WikiHow is a how-to database with a wide following and community as well as expert- and peer-review, the article texts document objects that are used and acted on in an activity. We therefore asked crowdworkers from the Upwork platform to collect articles; for robustness, we had five articles collected for each activity. We used a chunking model [79] to extract noun phrases from the article text, then we manually filtered the noun phrases into tangible objects.

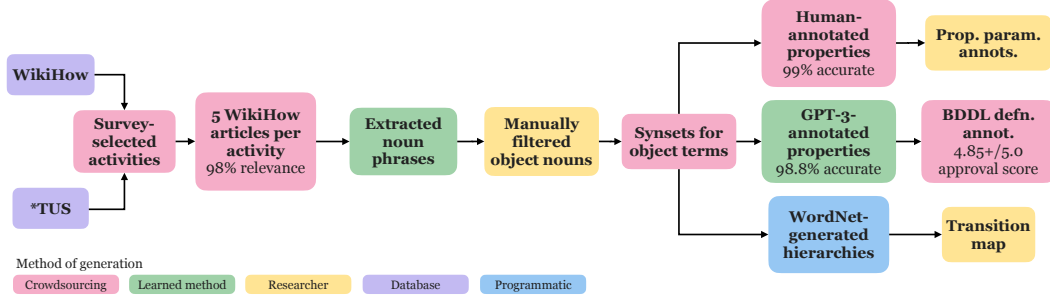


Figure A.2: **Flow chart of the annotation pipeline:** activity selection, activity article collection, object list extraction, object property and parameter annotation, BDDL annotation.

Synsets for object terms and WordNet hierarchy generation: After obtaining an object space for each activity, we take their union and crowdworkers match each one to a WordNet [57] synset. This eliminates word-sense ambiguity in the knowledge base and creates a hierarchical structure in the overall object space via the WordNet hierarchy. This process yielded 1,538 total WordNet leaf synsets. Based on various property annotations and needs of the activity set, there are also 1,426 custom synsets for a total of 2,964 leaf synsets.

Property annotations: Each object that is a leaf-level synset of the WordNet hierarchy is associated with the set of object properties in BEHAVIOR-1K, each of which is fully simulatable in OMNIGIBSON (full list of properties in Table A.1). The properties define which predicates can apply to the object - e.g. an object having the `cookable` property means it can be cooked and not cooked. This association is done task-agnostically because objects/states that are irrelevant or undesirable for a specific activity will still provide important learning signals. All properties that apply to many of the 2,964 leaf synsets and therefore require a large-scale annotation are done by either GPT-3 [58] or crowdworkers. GPT-3 is used for all properties (total of eight) for which the Hamming distance and false-positive rate for a sample compared to a human-annotated ground-truth are both less than 10%. Human annotation is used for five more properties. The remaining properties are either sensitive to the simulator implementation and are therefore annotated manually, or can be inferred from the other annotations and are determined analytically.

This annotation is done only for leaf-level synsets, then properties of higher-level synsets are inferred from the leaf-level annotations. This prevents definition unsolveability. In more detail: as shown in Fig. A.2, the hierarchical structure is applied to the object space. This allows activity definition annotators to refer to them instead of leaf-level synsets; for example, a `PuttingAwayGroceries` definition can have “five `edible_fruit.n.01s`” rather than “two `apple.n.01s` and three `banana.n.01s`”, allowing more variation in activity instantiation because far more object models are valid. However, 3-D object models are generally only attached to leaf- or near-leaf-level synsets, meaning that a definition may have a higher-level synset with various predicates attached to it, and at simulation time the object model (associated with one of the synset’s descendants) must have those states simulated. A problem might occur when e.g. `container.n.01` is annotated as `fillable` and the definition calls for a `container.n.01` to be filled with a liquid, but the actual model that gets sampled into the simulator to satisfy `container.n.01` is a cloth bag that doesn’t support being filled with a liquid. So to avoid this unsolveability, the properties of any non-leaf synset are exactly the *intersection of all its descendants’ properties*.

Object property	Annotation method	Prompt to annotator	Example objects
assembleable	manual	N/A	desk.n.01, table.n.02
breakable	human	Mark if the object can be broken into smaller pieces by a human dropping it on the floor without a tool.	wine_bottle.n.01, room_light.n.01
cloth	manual	N/A	hammock.n.02, canvas.n.01
coldSource	GPT-3	Is [object] a source of cold?	refrigerator.n.01, ice.n.01
cookable	GPT-3	Can a [object] be cooked?	biscuit.n.01, pizza.n.01
deformable	prog. (all softBody, cloth, rope)	N/A	tortilla.n.01, clay.n.01

diceable	prog. (all sliceable’s derivative synsets)	N/A	half__apricot.n.01, half__brisket.n.01
drapeable	prog. (all cloth, rope)	N/A	dress.n.01, bath__towel.n.01
fillable	manual	N/A	stockpot.n.01, bucket.n.01
fireSource	GPT-3	Is a [object] designed to create fire?	lighter.n.01, sparkler.n.01
flammable	human	Mark if the object can catch fire (i.e. burn with a flame).	candle.n.01, mail.n.01
foldable	prog. (all cloth, softBody)	N/A	tortilla.n.01, jean__jacket.n.01
freezable	prog. (all heatable)	N/A	olive__oil.n.01, ginger__beer.n.01
heatable	prog. (all rigid bodies)	N/A	oil.n.01, fish__knife.n.01
heatSource	GPT-3	Is a [object] a source of heat?	oven.n.01, toaster.n.01
liquid	GPT-3	Is a [object] a liquid?	gasoline.n.01, liquid__soap.n.01
macroPhysicalSubstance	manual	N/A	grated__cheese.n.01, blue- berry.n.02
meltable	manual	N/A	cheese.n.01, chocolate.n.02
microPhysicalSubstance	manual	N/A	brown__sugar.n.01, cinnamon.n.03
mixingTool	manual	N/A	putty__knife.n.01, teaspoon.n.02
needsOrientation	manual	N/A	cumin__shaker.n.01, salt__shaker.n.01
openable	human	Mark if the object is designed to be opened.	mixer.n.04, keg.n.01
particleApplier	manual	N/A	pepper__mill.n.01, deter- gent__atomizer.n.01
particleRemover	GPT-3	Can a [object] absorb liquid?	scrub__brush.n.01, broom.n.01
particleSource	manual (on top of particleApplier)	N/A	sink.n.01
particleSink	manual (on top of particleRemover)	N/A	sink.n.01
rigidBody	manual	N/A	tomato.n.01, desk.n.01
rope	manual	N/A	ribbon.n.01, fairy__light.n.01
sliceable	GPT-3	Can a [object] be sliced easily by a human with a knife?	sweet__corn.n.01, sandwich.n.01
slicingTool	GPT-3	Can a [object] slice an apple?	blade.n.09, razor.n.01
softBody	manual	N/A	dough.n.01, pillow.n.01
substance	prog. (all liquid, vis.Subst., phys.Subst.)	N/A	water.n.06, milk.n.01
toggleable	human	The object can be switched between a finite number of <i>discrete</i> states and is designed to do so.	hot__tub.n.01, light__bulb.n.01
unfoldable	prog. (all foldable)	N/A	tissue.n.02, foil.n.01
visualSubstance	manual	N/A	coriander.n.02, cocoa__powder.n.01
waterCook	manual (special case of cookable)	N/A	chickpea.n.01, white__rice.n.01

Table A.1: Properties annotated for BEHAVIOR-1K for each object category and included in the BEHAVIOR-1K DATASET knowledge base

Property parameter annotations: BDDL separates objects and object states (i.e. terms and predicates), but simulating realistic “cooking” or “filling” requires information for object-object property tuples, not just objects or object properties alone. For example, in the real world, an apple.n.01 and a chicken.n.01 do not become cooked at the same temperature, so simply knowing they are both cookable is insufficient. BEHAVIOR-1K DATASET therefore includes manual annotation of several property parameters that are (object, object property) specific, such as cookTemperature for all cookable objects.

Examples of object-property pairs and parameters:

- Temperature required for cookable object to go from (cooked object) = False to (cooked object) = True
 - crab.n.05: must reach **63°C**
 - squash.n.02: must reach **58°C**
 - meatball.n.01: must reach **63°C**
 - chicken__leg.n.01: must reach **74°C**
- Temperature generated by heatSource object. For toggleable heatSources, this may require toggledOn(object) = True
 - toaster__oven.n.01: generates **204°C**
 - ember.n.01: generates **1093°C**

- `hand_blower.n.01`: generates **45°C**
- `coffee_maker.n.01`: generates **93°C**

Transition rules: There are many activities that involve complex chemical and physical processes that are beyond the capability of the state-of-the-art simulation technology. For example, blending different fruits into a smoothie or sanding a rusted surface are extremely difficult to simulate, but the agent actions to complete these tasks are still within reach, e.g. placing the fruits inside a blender. Therefore, in order to support these type of activities, we create a set of transition rules that will be used by OMNIGIBSON to bypass the underlying physics but still produce visually realistic physical transitions, e.g. smoothie particles being generated inside the blender after the blender is turned on.

Examples of transition rules:

- Composition and decomposition of objects
 - Transition rule used in make a strawberry slushie
 - **Inputs:** `strawberry.n.01`, `ice.n.01`, `lemon_juice.n.01`, `agave.n.01`
 - **Transition machine:** `blender.n.01`
 - **Outputs:** `smoothie.n.01`
 - Transition rule used in make gazpacho
 - **Inputs:** `basil.n.03`, `salt.n.02`, `black_pepper.n.02`, `tomato_juice.n.01`, `cucumber.n.02`, `water.n.06`, `lemon_juice.n.01`
 - **Transition machine:** `saucepan.n.01`
 - **Outputs:** `gazpacho.n.01`
- Realistic cleaning rules
 - Transition rule used in `CleanTheExteriorOfYourGarage`
 - **Substance covering object:** `paint.n.01` or `spray_paint.n.01`
 - **Objects needed to remove:** `particleRemover` saturated with (`solvent.n.01` or `acetone.n.01`)
 - Transition rule used in `CleanYourRustyGardenTools`
 - **Substance covering object:** `rust.n.01` or `patina.n.01` or `incision.n.01` (simulated as particles but exposed to annotators as unary scratched predicate) or `tarnish.n.01` (simulated as particles but exposed to annotators as unary tarnished predicate)
 - **Objects needed to remove:** `emery_paper.n.01` or `whetstone.n.01`

Activity definitions in BDDL: Finally, the object spaces and the relevant properties and predicates they enable are offered to lay annotators to generate BDDL definitions. Annotators build activity definitions using an annotation interface that includes a visual version of BDDL [27] (details on new features in BDDL are in Sec. C). The annotation interface enforces the requirements needed to make definitions logically solvable and well-formed. We collect one definition per activity for a total of 1,000 BDDL activity definitions.

Listings 1, 2, and 3 show examples of BEHAVIOR-1K activity definitions, while Listing 4 and 5 show examples of BEHAVIOR-100 definitions. We see that while the numbers of objects and literals are similar, speaking to the fact that both benchmarks have reached similar scale and detail for the activities they have, BEHAVIOR-1K has far larger and more detailed activity distribution. The `BakingSugarCookies` activity involves transition rules that turn ingredients in `:init` to scones in `:goal`, a process that will require the listed mixer and oven in OMNIGIBSON. By contrast, cooking in BEHAVIOR-100 was limited to single objects transitioning from `not cooked` to `cooked`; other benchmarks are similar or lack cooking entirely. `CleanYourLaundryRoom` involves specific cleansing agents to clean mold, whereas BEHAVIOR-100 cleaning tasks only had two types of “dirtiness” (stained and dusty) and the only rule was to use water in the case of stained. `CleanTheBottomOfAnIron` also shows rust-specific cleaning (requiring emery paper).

	Article Collection	Object Extraction	Human Properties	GPT-3 / Machine Properties
Accuracy (Approval Rate)	0.974	0.968	0.990	0.988
F1-score	0.984	0.990	0.912	0.930
False Discovery Rate	0.026	0.032	0.031	0.029
False Positive Rate	N/A	N/A	0.002	0.003

Table A.2: Quality check results for object and object property annotations: for each stage in the annotation pipeline, report human verification results. The F1-score is $\frac{2}{\text{precision} + \text{recall}}$. The False Discovery Rate is $\text{FDR} = \text{FP}/(\text{TP} + \text{FP})$ and the False Positive Rate $\text{FPR} = \text{FP}/(\text{TN} + \text{FP})$. The high accuracy and F1-scores and low FDRs and FPRs show that our annotation results are of high quality.

	Activity Definition			
	Question 1	Question 2	Question 3	Question 4
Average Rating	4.875	4.942	4.967	4.975
Standard Deviation	0.331	0.234	0.364	0.156

Table A.3: Quality check results for activity definitions. Q1: Are the objects listed relevant to the definition of the task?, Q2: Do the initial placements/locations of objects make sense?, Q3: Are the actions to perform (“goals”) relevant to the definition of the activity?, Q4: Is this definition reasonable?

Each activity definition was evaluated on a scale of 1-5 for each of the questions, and the average score and standard deviation are presented. The increased granularity still reflects the high quality of our results.

B.1 Quality assessment of BEHAVIOR-1K annotations

Our quality control investigation shows us that all labeling done by crowdworkers and GPT-3 is of the highest quality. Five crowdworkers with an extensive background in data labeling for large machine learning projects, coding, or data verification affirmed the results from earlier crowdworkers. The accuracies for all the labeling tasks were all above 96%, the F1 scores were above 91% and the false positive and false discovery rates were between 2-3% as shown in Table A.2. We noticed that the Synset Verification is a bit lower in accuracy than the other labeling tasks. This may be due to subtleties in acceptable synset definitions: for example, a “reasonably narrow hypernym” of a word not found in WordNet [57] is acceptable, such as a “dispenser” for a “soap dispenser”, but there may be gray areas regarding what is considered “reasonably narrow” (e.g. would a “hand tool” be reasonable as a replacement for a “soap dispenser”?)

The activity definition process was evaluated with more granularity using a Likert scale and an array of questions. We found that the response values had consistently high averages (very close to the maximum score of 5) and low standard deviations as shown in Table A.2. We also found no significant discrepancy for activity definition scores across topics (e.g. tasks related to “cleaning”), showing that the BDDL definitions were rated uniformly across the span of activity categories.

C New BDDL features

BEHAVIOR-1K uses an expanded version of the BDDL used in BEHAVIOR-100 [27], in order to support the new set of diverse activities. There are three new features here: 1) representation of substances, 2) three-valued predicates, and 3) composition and decomposition of objects.

Representation of substances: BEHAVIOR-1K introduces *substances*, objects that are arbitrarily subdivideable and do not obey clear instance boundaries such as `water.n.06` or `flour.n.01`. The lack of instance boundaries is difficult with traditional PDDL/BDDL: for example, if there are two bottles of orange juice where the juice inside one is called `orange_juice.n.01_1` and the juice inside the other is called `orange_juice.n.01_2`, any mixing of the particles makes it near-impossible for the agent to satisfy a `:goal` condition pertaining to an instance. Even with quantification, a condition like `exists (orange_juice.n.01) (filled orange_juice.n.01 glass.n.01_3)` is still unfair: if the agent mixes particles from the two different `orange_juice` instances such that together they filled `glass.n.01_3` but neither instance alone has enough particles in `glass.n.01_3` to fill it, the `:goal` will not be met.

We simply enforce that there is up to one instance of any substance in a definition. The annotator can still control quantity by spawning it in as many containers as desired. Unlike labeling every particle separately, this maintains a compact representation. This does not allow for some of the

substances to be different from some (e.g. some of the orange juice is cold and some is hot), but we consider this an acceptable limitation.

Furthermore, particle-based objects can be computationally expensive to simulate. When an annotator uses `orange_juice.n.01` only as a container of orange juice and not actually the particles (e.g. in a `PuttingAwayGroceries` activity), this becomes a waste of computational resources. Therefore, we introduce a custom container synset for every substance that has the same properties and WordNet hierarchy structure as `bottle.n.01` and instruct annotators to use it if all they want is the container.

Three-valued predicates: A common issue in BEHAVIOR-100 activities is that success on predicates involving naturally continuous-valued quantities is sudden and arbitrary: cracking a window a little bit suddenly changes it from `not open` to `open`, and the activity from `undone` to `done` – even though the window is not a typical human conception of “open”. PDDL 2.1 [80] has numerical fluents and derived predicates, which offer continuous states. However, this granularity may not be crucial to the high-level activities in BEHAVIOR-1K, and the concept is also difficult to communicate to lay annotators.

We therefore treat certain predicates as three-valued by having two Boolean predicates where the negations are the same, such as `filled` and `empty` (rather than just `not filled`). The annotators still see one Boolean predicate, in this case `filled`, and negate it to say the opposite, but we assume that when they negate, they mean something decisively empty. This is a strong assumption but generally safe as people tend not to add expressions that seem like common sense. Wherever we see the predicate negated in the definition, we switch it out with the other predicate. This applies to `filled/empty`, `open/closed`, and `folded/unfolded`.

To ensure that the definition is logically the same after the switch, the underlying BDDL implementation converts the definition using De Morgan’s Law such that all negations are only applied to atomic formulae before the switch occurs.

Composition and decomposition of objects: In standard PDDL/ BDDL, the objects in `:objects` are assumed to persist throughout the activity. There is no concept of creating a new object that did not appear in the `:init` or explicitly destroying an object.

To enable our transition rules that involve turning some objects into others, we require a representation that will provide the desired information unambiguously. The `:objects` section cannot be inferred simply from a `:goal` that has new objects in it, because `:goal` is not exhaustive. We therefore introduce the future predicate, used in `:init` on all objects that do not appear in the scene when the agent enters, but must be present for the `:goal` to be satisfied. All objects in future predicates appear in `:objects` and they cannot appear in any other literals in `:init`. This approach takes inspiration from [81], which updates a reference to an object (e.g. a wall) as it keeps changing form as more sub-objects (e.g. blocks) are added to it.

D Scene and Object Models

In this section, we provide more details about the object and scene models presented in BEHAVIOR-1K DATASET, including the selection, modeling, and annotation processes.

The survey outlined in Sec. 2 provided us with a list of 1000 activities that humans prefer robots to perform. However, these activities are not restricted to a unique type of scene (e.g., houses). We chose scene types necessary to cover the BEHAVIOR-1K activities, including eight scene types: houses (15), houses with gardens (8), hotels (3), offices (5), grocery stores (4), generic halls (4), restaurants (6), and schools (5) (see Table A.4). These scenes cover activities that require a specific type (e.g., shopping, cooking, restocking) and activities that are generic and could happen in multiple scene types (e.g., cleaning). We also annotated how many activities could be performed on each type to guide us on how many instances of each scene type should we include in BEHAVIOR-1K DATASET. The main type of scene is still households: we improved 15 household scenes from BEHAVIOR-100 and annotated it further with light sources, new textures, etc. We then acquired additional instances of each scene type from online marketplaces such as TurboSquid [82].

Several of the available scene models in the marketplaces did not contain all the necessary rooms to perform the natural activities in BEHAVIOR-1K, e.g., restaurants did not include the kitchen, offices did not include the restrooms, or houses did not include the gardens. We collected separate models for those and contracted professional 3D designers to connect them.

Scene models were acquired with the necessary object models. However, they were not enough to cover the objects required by the activities in BEHAVIOR-1K. We acquired additional models to support the activities, as provided by the activity annotation process described in Sec. B, totaling 9,000+ object models from 1,900+ categories. The diversity of the object categories included in the dataset can be observed in Fig. A.7.

As provided by the 3D vendors, the scene and object models cannot be used directly for the simulation of the 1000 activities in BEHAVIOR-1K in OMNIGIBSON due to 1) lack of category annotation, 2) lack (or incorrect) of light sources in scenes, 3) incorrect part segmentation, 4) lack of articulation, 5) missing interactive elements like buttons, 6) lack of a unified canonical frame orientation for sampling, and 7) poor physical properties for realistic simulation. We manually cleaned and annotated all scenes and objects to correct these elements.

We will publicly release all the scenes and models to be used by other researchers. The models will be encrypted and only be used within OMNIGIBSON in order to comply with the rights of the model authors and the vendors' agreement. The documentation of the annotation process as well as source code for the pipeline will similarly be released on our website, allowing users to easily import their own objects and scenes into OMNIGIBSON for use in BEHAVIOR-1K activities.

Scene Type	Scene Name	Obj. Cnt.	Syn. Cnt.	Rm. Cnt.	Room Types	Example Activities
BEHAVIOR-100 Houses	Beechwood_0_int	136	32	8	bathroom, corridor, dining_room, entryway, kitchen, living_room, private_office, utility_room	clean a hot water dispenser, freeze lasagna, clean batting gloves
	Beechwood_1_int	129	21	9	bathroom, bedroom, childs_room, closet, corridor, playroom, television_room	clean your kitty litter box, store baby clothes, cleaning pet bed
	Benevolence_0_int	21	10	4	bathroom, corridor, empty_room, entryway	laying clothes out, pack a pencil case, sweeping outside entrance
	Benevolence_1_int	74	22	5	corridor, dining_room, kitchen, living_room, storage_room	hanging blinds, sorting volunteer materials, wash baby bottles
	Benevolence_2_int	63	23	5	bathroom, bedroom, corridor	clean walls, washing fabrics, folding clean laundry
	Ihlen_0_int	68	21	7	bathroom, corridor, dining_room, garage, living_room, storage_room	de-clutter your garage, sorting household items, set up a home office in your garage
	Ihlen_1_int	147	27	9	bathroom, bedroom, corridor, dining_room, kitchen, living_room, staircase	clean jewels, clean green beans, hanging up curtains
	Merom_0_int	82	27	6	bathroom, childs_room, living_room, playroom, storage_room, utility_room	changing dog's water, wash a bra, wash a wool coat
	Merom_1_int	123	28	8	bathroom, bedroom, childs_room, corridor, dining_room, kitchen, living_room, staircase	store an uncooked turkey, drying table, clean flip flops
	Pomaria_0_int	71	22	6	bathroom, bedroom, corridor, private_office, television_room	prepare sea salt soak, clean sheets, dispose of medication
	Pomaria_1_int	89	24	6	bathroom, corridor, kitchen, living_room, pantry_room, utility_room	store brownies, clean a book, baking sugar cookies
	Pomaria_2_int	42	18	3	bathroom, bedroom, corridor	replacing screens, clean baby toys, putting up posters
	Rs_int	72	32	5	bathroom, bedroom, entryway, kitchen, living_room	dispose of a pizza box, putting food in fridge, putting out condiments
	Wainscott_0_int	187	35	9	bathroom, bedroom, dining_room, kitchen, living_room	make microwave popcorn, make frozen lemonade, make cake mix
	Wainscott_1_int	150	26	10	bathroom, bedroom, corridor, exercise_room, playroom, private_office, utility_room	decorating for religious ceremony, stash snacks in your room, disinfect laundry

BEHAVIOR-100 Houses+Garden	Beechwood_0_garden	335	45	9	bathroom, corridor, dining_room, entryway, garden, kitchen, living_room, private_office, utility_room	tidy your garden, opening doors, wash goalkeeper gloves
	Merom_0_garden	197	51	8	bathroom, childs_room, garden, living_room, playroom, sauna, storage_room, utility_room	clean your kitty litter box, clean a glass windshield, hang icicle lights
	Pomaria_0_garden	260	49	7	bathroom, bedroom, corridor, garden, private_office, television_room	putting up Christmas lights outside, prepare a hanging basket, clean a patio
	Rs_garden	185	47	6	bathroom, bedroom, entryway, garden, kitchen, living_room	cleaning driveway, clean a long-board, roast nuts
	Wainscott_0_garden	712	61	10	bathroom, bedroom, dining_room, garden, kitchen, living_room	clean an espresso machine, clearing food from table into fridge, painting porch
New Houses	house_single_floor	1375	137	23	bathroom, bedroom, childs_room, closet, corridor, dining_room, empty_room, entryway, garden, kitchen, living_room, sauna, utility_room	turning out all lights before sleep, iron curtains, packing hobby equipment
	house_double_floor_lower	304	79	6	bathroom, corridor, garage, garden, kitchen, living_room	wash towels, clean a fish, clean brass
	house_double_floor_upper	325	78	5	bathroom, bedroom, television_room	clean a saxophone, taking down curtains, clean walls
Grocery Stores	grocery_store_asian	3402	79	2	bathroom, grocery_store	buy alcohol, buy boxes for packing, buy a microwave oven
	grocery_store_cafe	6994	71	4	bar, bathroom, dining_room, grocery_store	defrost meat, clean reusable shopping bags, buy a good avocado
	grocery_store_convenience	1889	82	2	bathroom, grocery_store	washing windows, picking up prescriptions, buy food for vegetarians
	grocery_store_half_stocked	3804	51	2	bathroom, grocery_store	buying gardening supplies, buy basic garden tools, buy boxes for packing
Halls	hall_arch_wood	78	11	2	bathroom, empty_room	clean walls
	hall_train_station	141	15	2	bathroom, empty_room	clean cement
	hall_glass_ceiling	116	20	2	bathroom, empty_room	preparing food for a fundraiser
	hall_conference_large	3372	26	2	bathroom, conference_hall	distributing event T-shirts
Hotels	hotel_gym_spa	322	43	9	bathroom, corridor, gym, hammam, locker_room, sauna, spa	turning on the hot tub, adding chemicals to hot tub, clean a sauna
	hotel_suite_large	218	51	2	bathroom, bedroom	clean vinyl shutters, cleaning computer, clean white marble
	hotel_suite_small	48	28	2	bathroom, bedroom	clean cork mats, putting out clean towels, clean a shower
Offices	office_bike	479	49	3	bathroom, break_room, shared_office	set up a webcam, set up two computer monitors, clean an office chair
	office_cubicles_left	787	44	12	bathroom, conference_hall, copy_room, corridor, lobby, meeting_room, private_office, shared_office	clean up your desk, clean a LED screen, laying out snacks at work
	office_cubicles_right	406	37	11	bathroom, conference_hall, copy_room, corridor, lobby, meeting_room, private_office, shared_office	making photocopies, clean a LED screen, clean a keyboard
	office_large	1151	46	24	bathroom, break_room, conference_hall, copy_room, corridor, lobby, meeting_room, phone_room, private_office, shared_office	emptying trash cans, putting meal in fridge at work, brewing coffee
	office_vendor_machine	225	45	4	bathroom, break_room, meeting_room, shared_office	clean a computer monitor, make the workplace exciting, dispose of glass
Restaurants	restaurant_asian	1221	56	3	bathroom, dining_room, kitchen	grill vegetables, cook tofu, clean clams
	restaurant_brunch	1096	70	4	bar, bathroom, dining_room, kitchen	stock a bar, cook squid, washing vegetables
	restaurant_cafeteria	292	45	3	bathroom, dining_room, kitchen	clean a popcorn machine, store coffee beans or ground coffee, roast meat
	restaurant_diner	174	39	3	bar, bathroom, dining_room	sweeping floors, installing smoke detectors, clean a lobster
	restaurant_hotel	1080	81	4	bathroom, dining_room, kitchen, lobby	clean eggs, setting table for coffee, clean a pizza stone
	restaurant_urban	1368	90	4	bar, bathroom, dining_room, kitchen	cook pumpkin seeds, putting roast in oven, thaw frozen fish
Schools	school_biology	717	53	4	bathroom, biology_lab, corridor	clean an eraser, turning sprinkler off, clean a glass pipe

school_chemistry	890	69	4	bathroom, chemistry_lab , corridor	clean clear plastic, clean an eraser, clean a whiteboard
school_comp_lab_infirmery	828	61	6	bathroom, computer_lab, corridor, infirmery	clean a computer monitor, prepare an emergency school kit, clean a keyboard
school_geography	556	42	5	bathroom, classroom, corridor	clean gold, clean a glass pipe, clean an eraser
school_gym	853	36	7	bathroom, corridor, gym , locker_room	clean a dirty baseball, dispose of glass, wash towels

Table A.4: Statistics and information for each of 50 scenes in B1K: scene type, name, number of objects, unique synsets and rooms, room types, and example activities

E Simulation

E.1 Extended Object States and Logical Predicates in OMNIGIBSON

OMNIGIBSON extends the infrastructure of extended object states and logical predicates from iGibson 2.0 [59] to accommodate the diversity and realism of BEHAVIOR-1K.

E.1.1 Extended object states associated with object category properties

For computational efficiency purposes, not all extended object states need to be maintained and updated for every object. For example, cookable objects like apples need to keep track of their Temperature, but probably tables don’t need to, at least for the purpose of simulating BEHAVIOR-1K activities. Hence, given the object category property annotation from BEHAVIOR-1K DATASET (see Table A.1), OMNIGIBSON selectively keeps track of a subset of the extended states for objects of each category. The mapping from the object category property to the required object states can be found in Table A.5.

Object category property	Required extended object states
cookable	MaxTemperature, Temperature
overcookable	MaxTemperature, Temperature
freezable	Temperature
flammable	Temperature
heatable	Temperature
metable	Temperature
soakable	SoakedLevel
toggleable	ToggledState
sliceable	SlicedState
breakable	BrokenState
heatSource	ToggledState
fireSource	ToggledState
coldSource	ToggledState
waterSource	ToggledState

Table A.5: Extended object states associated with object category properties. Adapted from [59].

E.1.2 Object model properties

We also need to perform physical and semantic annotation for each object model so that they can be realistically simulated and support the evolution of extended object states. Some of the physical properties can be programmatically generated from the 3D assets, such as Shape, KinematicStructure and StableOrientations, while others need to be annotated (included in BEHAVIOR-1K DATASET), such as Weight. Type of an object model, which is also derived from the object category property annotation from BEHAVIOR-1K DATASET (see Table A.1), determines how the object will be simulated in OMNIGIBSON. For example, cloths and fluids will be simulated using underlying particle systems. Also, some of the object category property requires additional semantic annotation for each model of that category, e.g. for waterSource objects, WaterSourceLink is required to indicate the exact location to generate water. An exhaustive list of all the object model properties can be found in Table A.6.

Object model property	Relevant object category property	Description
Shape		Model of the 3D shape of each link of the object
Weight		Weight of the object
CenterOfMass		Mean position of the matter in the object
MomentOfInertia		Resistance of the object to change its angular velocity
KinematicStructure		Structure of links and joints connecting them in the form of URDF (non-articulated objects are composed of one link)
StableOrientations		A list of stable orientations assuming the object is placed on a flat surface, computed using a 3D geometry library
HeatSourceLink	heatSource	Virtual (non-colliding) fixed link that generates heat
FireSourceLink	fireSource	Virtual (non-colliding) fixed link that generates fire
CleaningToolLink	cleaningTool	Fixed link that needs to contact dirt particles for the tool to clean them
WaterSourceLink	waterSource	Virtual (non-colliding) fixed link that generates water
WaterSinkLink	waterSource	Virtual (non-colliding) fixed link that absorbs water
TogglingLink	toggleable	Virtual (non-colliding) fixed link that changes the toggled state of the object when contacted
SlicingLink	slicingTool	Fixed link that changes the sliced state of another object if it contacts it with enough force
RelevantJoints	openable	List of joints that are relevant to indicate whether an object is open
AttachmentKeypoints	assembleable	List of keypoint locations that will be connected with those of other objects when they are close enough
ClothKeypoints	foldable, unfoldable	List of keypoint locations are used to determine if the object is folded (if they are close enough) or unfolded (if they are far enough)
ContainerVolume	fillable	Virtual (non-colliding) fixed link that represents the inner volume of a fillable container.
Type	cloth, deformable, liquid, rope, physicalSubstance, visualSubstance	Type of the object, e.g. rigid body, fluid, physical/visual substance, cloth, deformable, which determines how it will be simulated in OMNIGIBSON

Table A.6: Permanent object model properties. Adapted from [59].

E.1.3 Object states

While the object properties mentioned above stay the same during simulation (i.e. an apple is always cookable and a sink always have the same `WaterSourceLink` defined in its local frame), the object states could change. The underlying physics engine handles the kinematic state changes, such as `Pose`, `AABB`, `JointStates`, `ParticlePositions` for fluids and cloths, etc. On top of these, OMNIGIBSON handles the non-kinematic state changes that are essential for logical predicates (described in the next subsection). For example, OMNIGIBSON update the `Temperature` of objects at every simulation step by checking if they are near/inside any `heatSource` or `coldSource`. An exhaustive list of all the object states can be found in Table A.7.

E.1.4 Logical predicates as checking functions

For each logical predicate that is relevant for BEHAVIOR-1K activities, we define a checking function that maps a given physical state (kinematic and non-kinematic) into a binary logical state that BDDL operates on. For example, `OnTopOf` is based on the pose of two objects and their contact information whereas `Cooked` is based on the `Temperature`. The details of the checking functions for all the logical predicates can be found in Table A.8.

E.1.5 Logical predicates as sampling functions

For each logical predicate that is relevant for BEHAVIOR-1K activities, we also define a generative function that samples a valid physical state given a binary logical state. This functionality is essential for infinite activity initialization. For example, if the initial conditions of the activity require a plate to be `OnTopOf` a dining table, there are an infinite number of exact poses of the plate that can satisfy this condition. Our generative functions will find a valid solution and physically put the plate on top of the table. Other examples include setting the `Temperature` of an object to make it `Frozen` or the joint configuration of an object to make it `Open`. The details of the sampling functions for all the logical predicates can be found in Table A.8.

Object State	Description and Update Rules
Pose	6 DoF pose (position and orientation) of the object in world reference frame, updated by the underlying physics engine.
AABB	Axis-aligned bounding box (coordinates of two opposite corners) of the object in the world reference frame, updated by the underlying physics engine.
JointStates	State of all internal DoFs of the (articulated) object for the structure defined by Kinematic-Structure , updated by the underlying physics engine.
ParticlePositions	Positions of all the underlying particles for cloth and fluid, updated by the underlying physics engine.
InContactObjs	List of all objects in physical contact with the object, updated by the underlying physics engine.
ConnectedObjs	List of all objects that are connected to the object, either via a fixed joint for rigid bodies or via an attachment for cloth and deformables.
InSamePositiveVerticalAxisObjs	List of all objects in the positive vertical axis drawn from the object’s center of mass, updated by shooting a ray upwards in the positive z-axis and gather the objects hit by the ray.
InSameNegativeVerticalAxisObjs	List of all objects in the negative vertical axis drawn from the object’s center of mass, updated by shooting a ray downwards in the negative z-axis and gather the objects hit by the ray.
InSameHorizontalPlaneObjs	List of all objects in the horizontal plane drawn from the object’s center of mass, updated by shooting a number of ray in the x-y plane and gather the objects hit by the rays.
Temperature, T	Object’s current temperature in $^{\circ}\text{C}$, updated by detecting if the object is affected by any heat source or heat sink.
MaxTemperature, T_{max}	Maximum temperature of the object reached historically during this simulation run, updated by keeping track of all the Temperature in the history.
SoakedLevel, w	Amount of liquid absorbed by the object corresponding to the number of liquid particles contacted, updated by detecting if the object is in contact with any liquid particle. This is maintained for every type of liquid separately.
CoveredLevel, c	Amount of visualSubstance that covers the object, updated by detecting if the particles of the visualSubstance are in contact with anything that can potentially remove them from the object, e.g. cleaningTool . This is maintained for every type of visualSubstance separately.
ToggledState, TS	Binary state indicating if the object is currently on or off, updated by detecting if the agent is in contact with the TogglingLink .
SlicedState, SS	Binary state indicating whether the object has been sliced (irreversible), updated by detecting if the object is in contact with any SlicingTool that exerts a force above a certain threshold F_{sliced} . We assume as default force threshold of $F_{sliced} = 10\text{ N}$, a value that can be configured per object category and model.
BrokenState, BS	Binary state indicating if the object is broken, updated by detecting if the object has a contact force with any other object above a certain threshold F_{broken} , a value that can be configured per object category and model.

Table A.7: Object states maintained by OMNIGIBSON. Adapted from [59].

E.2 AMT Visual Realism Study

We conducted an Amazon Mechanical Turk (AMT) study to evaluate OMNIGIBSON’s relative visual realism compared to multiple other simulation environments. We selected 50 representative 1280×720 images from OMNIGIBSON, AI2-Thor, ThreeDWorld, Habitat 2.0, and iGibson 2.0, and shuffled them randomly into 50 groups of 5 images, where each group contained a unique image from each simulation environment. For each group, participants were asked to rank the images in terms of visual realism, assigning the most realistic images of the group 1, and subsequent images 2, ..., 5. Image shuffling was randomized between participants. When presenting our results, we invert the aggregated mean and standard deviation across 60 participants, such that a score of a 5 would represent the most visually realistic images.

Our criteria for selecting the images are the following: (a) we only included photos taken from *fully interactive* scenes for a fair comparison, and (b) rendering must come from within the simulation environment, without customized tuning (i.e. new users can expect this visual quality off-the-shelf with minimal adjustment).

E.3 Visual Modalities

OMNIGIBSON provides diverse perceptual modules to capture realistic sensor modalities from an agent’s perspective, including RGB, Depth, Semantic Segmentation, Normal, and Optical Flow images, in addition to non-visual modalities such as proprioception and LiDAR scans (Fig. A.8). OMNIGIBSON also provides varying abstraction levels for agent action spaces, including low-level control, assistive manipulation, and primitive skill execution. Overall, these modules are intended to

Predicate	Description
InsideOf(o_1, o_2)	Object o_1 is inside of object o_2 if we can find two orthogonal axes crossing at o_1 center of mass that intersect o_2 collision mesh in both directions.
OnTopOf(o_1, o_2)	Object o_1 is on top of object o_2 if $o_2 \in \text{InSameNegativeVerticalAxisObjs}(o_1) \wedge o_2 \notin \text{InSamePositiveVerticalAxisObjs}(o_1) \wedge \text{InContactWith}(o_1, o_2)$, where $\text{InSamePositive/NegativeVerticalAxisObjs}(o_1)$ is the list of objects in the same positive/negative vertical axis as o_1 and $\text{InContactWith}(o_1, o_2)$ is whether the two objects are in physical contact.
NextTo(o_1, o_2)	Object o_1 is next to object o_2 if $o_2 \in \text{InSameHorizontalPlaneObjs}(o_1) \wedge l2(o_1, o_2) < t_{\text{NextTo}}$, where $\text{InSameHorizontalPlaneObjs}(o_1)$ is a list of objects in the same horizontal plane as o_1 , $l2$ is the L2 distance between the closest points of the two objects, and t_{NextTo} is a distance threshold that is proportional to the average size of the two objects.
InContactWith(o_1, o_2)	Object o_1 is in contact with o_2 if their surfaces are in contact in at least one point, i.e., $o_2 \in \text{InContactObjs}(o_1)$.
ConnectedWith(o_1, o_2)	Object o_1 is connected with o_2 if $o_2 \in \text{ConnectedObjs}(o_1)$.
Under(o_1, o_2)	Object o_1 is under object o_2 if $o_2 \in \text{InSamePositiveVerticalAxisObjs}(o_1) \wedge o_2 \notin \text{InSameNegativeVerticalAxisObjs}(o_1)$.
OnFloor(o_1, o_2)	Object o_1 is on the room floor o_2 if $\text{InContactWith}(o_1, o_2)$ and o_2 is of Room type.
Open(o)	Any joints (internal articulated degrees of freedom) of object o are open. Only joints that are relevant to consider an object Open are used in the predicate computation, e.g. the door of a microwave but not the buttons and controls. To select the relevant joints, object models of categories that can be Open undergo an additional annotation that produces a RelevantJoints list. A joint is considered open if its joint state q is 5% over the lower limit, i.e. $q > 0.05(q_{\text{UpperLimit}} - q_{\text{LowerLimit}}) + q_{\text{LowerLimit}}$.
Cooked(o)	The temperature of object o was over the cooked threshold, T_{cooked} , and under the burnt threshold, T_{burnt} , at least once in the history of the simulation episode, i.e., $T_{\text{cooked}} \leq T_o^{\text{max}} < T_{\text{burnt}}$. We annotate the cooked temperature T_{cooked} for each object category that can be Cooked.
Burnt(o)	The temperature of object o was over the burnt threshold, T_{burnt} , at least once in the history of the simulation episode, i.e., $T_o^{\text{max}} \geq T_{\text{burnt}}$. We annotate the burnt temperature T_{burnt} for each object category that can be Burnt.
OnFire(o)	The temperature of object o is above the on-fire threshold, T_{onfire} , i.e., $T_o \leq T_{\text{onfire}}$. We assume as default on-fire temperature $T_{\text{onfire}} = 300^\circ C$, a value that can be adapted per object category and model.
Frozen(o)	The temperature of object o is under the freezing threshold, T_{frozen} , i.e., $T_o \leq T_{\text{frozen}}$. We assume as default freezing temperature $T_{\text{frozen}} = 0^\circ C$, a value that can be adapted per object category and model.
Heated(o)	The temperature of object o is above the heated threshold, T_{heated} , i.e., $T_o \leq T_{\text{heated}}$. We assume as default heated temperature $T_{\text{heated}} = 75^\circ C$, a value that can be adapted per object category and model.
Boiled(l)	The temperature of liquid l is above the boiling point, T_{boiled} , i.e., $T_o \leq T_{\text{boiled}}$. We assume as default boiling point $T_{\text{boiling}} = 100^\circ C$, a value that can be adapted per object category and model.
Soaked(o, l)	The soaked level w of the liquid l for the object o is over a threshold, w_{soaked} , i.e., $w \geq w_{\text{soaked}}$. The default value for the threshold is $w_{\text{soaked}} = 50$, (the object is soaked if it absorbs more than 50 liquid particles), a value that can be adapted per object category and model and per liquid type.
Filled(o, l)	Object o is filled by liquid l if the number of particles of l that is inside the ContainerVolume of o is above a certain threshold percentage of the total volume. The default value for the threshold is $w_{\text{filled}} = 0.5$.
Covered(o, s)	For visualSubstance s , check if the covered level c of s for the object o is over a threshold, c_{covered} , i.e., $c \geq c_{\text{covered}}$; for physicalSubstance s , check if the number of particles of s that are in contact with the object o is over the same threshold. The default value for the threshold is $c_{\text{covered}} = 50$ (50 substance particles), a value that can be adapted per object category and model, and per substance.
ToggledOn(o)	Object o is toggled on or off. It is a direct query of the object's extended state TS , the toggled state.
Sliced(o)	Object o is sliced or not. It is a direct access of the object's extended state SS , the sliced state.
Broken(o)	Object o is broken or not. It is a direct access of the object's extended state BS , the broken state.
Folded(o)	Object o is folded if its corresponding ClothKeypoints are sufficiently close to each other.
Unfolded(o)	Object o is unfolded if its corresponding ClothKeypoints are sufficiently far from each other.
Assembled(o)	Object o is assembled if all of its parts have been correctly connected: every pair of parts p_i and p_j are connected (or not) with each other (i.e. $\text{IsConnected}(p_i, p_j)$) in a specific way defined by each object model.
Hung(o_1, o_2)	Object o_1 is hung onto object o_2 if they are connected, $\text{IsConnected}(o_1, o_2)$.
Blended($o_1 \dots o_n$)	Objects o_1 to o_n are blended if they are in contact with each other, i.e. $\text{InContactWith}(o_i, o_j)$ for all pairs.
InFoVOfAgent(o)	Object o is in the field of view of the agent, i.e., at least one pixel of the image acquired by the agent's onboard sensors corresponds to the surface of o .
InHandOfAgent(o)	Object o is grasped by the agent's hands (i.e. assistive grasping is activated on that object).
InReachOfAgent(o)	Object o is within $d_{\text{reach}} = 2$ meters away from the agent.
InSameRoomAsAgent(o)	Object o is located in the same room as the agent.

Table A.8: **Logical Predicates:** Description of the checking functions. Adapted from [59].

Predicate	Sampling Mechanism
InsideOf(o_1, o_2)	Only $\text{InsideOf}(o_1, o_2) = \text{True}$ can be sampled. o_1 is randomly sampled within o_2 using a ray-casting mechanism adopted from [27]. o_1 is guaranteed to be supported fully by a surface and free of collisions with any other object except o_2 .
OnTopOf(o_1, o_2)	Only $\text{OnTopOf}(o_1, o_2) = \text{True}$ can be sampled. o_1 is randomly sampled on top of o_2 using a ray-casting mechanism adopted from [27]. o_1 is guaranteed to be supported fully by a surface and free of collisions with any other object except o_2 .
ConnectedWith(o_1, o_2)	Create a rigid joint between the two objects if they are both rigid bodies, or an attachment between them otherwise.
Under(o_1, o_2)	Only $\text{Under}(o_1, o_2) = \text{True}$ can be sampled. o_1 is randomly sampled on top of the floor region beneath o_2 using a ray-casting mechanism adopted from [27]. o_1 is guaranteed to be supported fully by a surface and free of collisions with any other object except the floor.
OnFloor(o_1, o_2)	Only $\text{OnFloor}(o_1, o_2) = \text{True}$ can be sampled. o_1 is randomly sampled on top of o_2 , which is the floor of a certain room, using the scene’s room segmentation mask. o_1 is guaranteed to be supported fully by a surface and free of collisions with any other object except o_2 .
Open(o)	To sample an object o with the predicate $\text{Open}(o) = \text{True}$, a subset of the object’s relevant joints (using the <code>RelevantJoints</code> model property) are selected, and each selected joint is moved to a uniformly random position between the openness threshold and the joint’s upper limit. To sample an object o with the predicate $\text{Open}(o) = \text{False}$, all of the object’s relevant joints (using the <code>RelevantJoints</code> model property) are moved to a uniformly random position between the joint’s lower limit and the openness threshold.
Cooked(o)	To sample an object o with the predicate $\text{Cooked}(o) = \text{True}$, the object’s <code>MaxTemperature</code> is updated to $\max(T_o^{\text{max}}, T_{\text{cooked}})$. Similarly, to sample an object o with the predicate $\text{Cooked}(o) = \text{False}$, the object’s <code>MaxTemperature</code> is updated to $\min(T_o^{\text{max}}, T_{\text{cooked}} - 1)$.
Burnt(o)	To sample an object o with the predicate $\text{Burnt}(o) = \text{True}$, the object’s <code>MaxTemperature</code> is updated to $\max(T_o^{\text{max}}, T_{\text{burnt}})$. Similarly, to sample an object o with the predicate $\text{Cooked}(o) = \text{False}$, the object’s <code>MaxTemperature</code> is updated to $\min(T_o^{\text{max}}, T_{\text{burnt}} - 1)$.
OnFire(o)	To sample an object o with the predicate $\text{OnFire}(o) = \text{True}$, the object’s <code>Temperature</code> is updated to a uniformly random temperature between $T_{\text{onfire}} + 10$ and $T_{\text{onfire}} + 50$. To sample an object o with the predicate $\text{OnFire}(o) = \text{False}$, the object’s <code>Temperature</code> is updated to $T_{\text{onfire}} - 1$.
Frozen(o)	To sample an object o with the predicate $\text{Frozen}(o) = \text{True}$, the object’s <code>Temperature</code> is updated to a uniformly random temperature between $T_{\text{frozen}} - 10$ and $T_{\text{frozen}} - 50$. To sample an object o with the predicate $\text{Frozen}(o) = \text{False}$, the object’s <code>Temperature</code> is updated to $T_{\text{frozen}} + 1$.
Heated(o)	To sample an object o with the predicate $\text{Heated}(o) = \text{True}$, the object’s <code>Temperature</code> is updated to a uniformly random temperature between $T_{\text{heated}} + 10$ and $T_{\text{heated}} + 50$. To sample an object o with the predicate $\text{Heated}(o) = \text{False}$, the object’s <code>Temperature</code> is updated to $T_{\text{heated}} - 1$.
Boiled(l)	To sample a liquid type l with the predicate $\text{Boiled}(l) = \text{True}$, the <code>Temperature</code> of all particles of l is updated to a uniformly random temperature between $T_{\text{boiled}} + 10$ and $T_{\text{boiled}} + 50$. To sample a liquid type l with the predicate $\text{Boiled}(o) = \text{False}$, the <code>Temperature</code> of all particles of l is updated to $T_{\text{boiled}} - 1$.
Soaked(o, l)	To sample an object o and a liquid type l with the predicate $\text{Soaked}(o, l) = \text{True}$, the object’s <code>SoakedLevel</code> w for l is updated to match the <code>Soaked</code> threshold of w_{soaked} . To sample an object o and a liquid type l with the predicate $\text{Soaked}(o, l) = \text{False}$, the object’s <code>SoakedLevel</code> w is updated to 0.
Filled(o, l)	To sample an object o and a liquid type l with the predicate $\text{Filled}(o, l) = \text{True}$, an appropriate number of particles of l are sampled inside the <code>ContainerVolume</code> of o so that they fill up enough of its volume ($\geq w_{\text{filled}} = 0.5$). To sample an object o and a liquid type l with the predicate $\text{Filled}(o, l) = \text{False}$, all particles of l that are inside the <code>ContainerVolume</code> of o (if any) are removed.
Covered(o, s)	To sample an object o and a substance s with $\text{Covered}(o, s) = \text{True}$, a fixed number of particles of s are randomly placed on the surface of o using a ray-casting mechanism adopted from [27]. To sample an object o and a visualSubstance s with $\text{Covered}(o, s) = \text{False}$, all particles of s is removed from o and the corresponding <code>CoveredLevel</code> is set to 0.
ToggledOn(o)	The <code>ToggledState</code> of the object is updated to match the required predicate value.
Sliced(o)	The <code>SlicedState</code> of the object is updated to match the required predicate value. Also, the whole object are replaced with the two halves, that will be placed at the same location and inherit the extended states from the whole object (e.g. <code>Temperature</code>).
Broken(o)	The <code>BrokenState</code> of the object is updated to match the required predicate value. Also, the whole object is broken down into pieces, that will be placed at the same location.

Table A.9: **Logical Predicates:** Description of the sampling functions. Adapted from [59].

be useful to the broad embodied AI research community, with the goal of accelerating breakthroughs on BEHAVIOR-1K.

E.4 Performance Benchmark

To evaluate the performance of OMNIGIBSON under different conditions, we performed rigorous speed test in two representative scenes (`Rs_int` and `house_single_floor`) with different number of objects on a single-GPU, single-process setup. We adopt the “idle” setup from Li et al. [59] and Szot et al. [26]: a robot is placed in the scene (except the last row “- Robot”), and stays still with zero velocity action. At each time step, the simulator runs the physics simulation, extended object state update, and transition machine update loop, and renders a 128×128 RGB image. We use action

Eval. Conditions	Scene	
	Rs_int (81 objs)	house_single_floor (621 objs)
Full Feature Set	24	11
- Fluid and Cloth	58	26
- Object State Update	77	55
- Robot	90	60

Table A.10: Benchmarking OMNIGIBSON performance: simulation steps per second (SPS, higher better) in two representative scenes with 81 and 621 objects, under different evaluation conditions.

time step of $t_a = \frac{1}{60}$ s and physics time step of $t_s = \frac{1}{60}$ s. Our benchmark runs on a Ubuntu machine with Intel(R) Core(TM) i7-10700K CPU @ 5.10GHz and one Nvidia GeForce RTX 3080 GPU in a single process setting. The results are summarized in Table A.10.

OMNIGIBSON runs at a comparable speed to previous works like iGibson 2.0 [59] under similar settings, but with much higher rendering quality thanks to ray-tracing. It maintains a reasonable speed even for a large scene with over 600 objects. OMNIGIBSON is also highly configurable and provides flexible interface for users to balance between simulation fidelity and speed given their use cases and research interests. If the user isn’t interested in fluid or cloth in their tasks, they can turn off these features to harvest performance speedup. Similarly, if the user is only interested in kinematics-only rearrangement tasks, or non-robotics embodied AI applications (e.g. a virtual camera), they can turn off object state update, or remove the robot, respectively. We are actively working on improving aspects that can provide simulation speedups, e.g. in areas like object sleeping, mesh simplification, more efficient object state update. Furthermore, since real-time ray-tracing and fluid/cloth simulation is still an active area of research, we expect the upcoming hardware and software advances from Nvidia will lead to significant performance improvements in these areas.

F Baselines Details

In this section, we include additional information about the baselines evaluated and analyzed in the main paper: the visuomotor control baseline and two variants of the baselines using action primitives, with and without the history of observations.

F.1 Network Architecture

For RL-Prim. and RL-Prim.Hist. that use PPO as the underlying RL algorithm, the architecture consists of a visual feature extractor that takes the egocentric visual observation as input and a state encoder neural network which takes whether the robot is grasping an object or not as input. The image input is normalized using a per-channel moving average. These two encodings are passed into an MLP module, which is then processed by a value head to predict the value for the given observations and an action head to produce a discrete action that corresponds to an action primitive executed on an object. The size of input images is $128 \times 128 \times 3$. The feature extractor is a sequential architecture of Conv-ReLU-MaxPooling-Flatten. MLP converts the feature into a 128-dimensional vector. The details of RL-Prim.’s network architecture is illustrated in Fig. A.11 (b). For RL-VMC that uses SAC as the underlying RL algorithm, we use the same feature extractor as RL-Prim.. RL-VMC consists of an actor network, a critic network, and a target critic network. All of them are MLPs with ReLU activation functions. Fig. A.11 (a) illustrates the details of RL-VMC’s network architecture. RL-VMC has the continuous action space and outputs low-level control actions directly.

F.2 Task settings

Fig. A.9 depicts the three activities we consider in our experiments:

- **StoreDecoration:** a tidying activity where the agent must pick up and store Halloween decorations into a cabinet operating articulated objects. Action space: navigate, pick, place, push. The goal is to store two pumpkins into a drawer (see Fig. A.9 (a)).
- **CollectTrash:** A collecting activity that requires the agent to gather empty bottles and cups and throw them into a trash bin. Action space: navigate, pick, place. The goal is to throw two bottles and two cups into a trash bin (see Fig. A.9 (b)).

Learning Rate	0.0003
Buffer Size	300
Batch Size	64
Discount (γ)	0.99
Soft Update Coefficient	0.005

Table A.11: Hyperparameters of SAC for the RL-VMC baseline

Learning Rate	0.0003
Buffer Size	300
Batch Size	64
Discount (γ)	0.99
GAE Parameter γ_{gae}	0.99
Clipping Parameter ϵ	0.2
Entropy Coeff c_1	0.0
VF Coeff c_2	0.5

Table A.12: Hyperparameters of PPO for the RL-Prim. and RL-Prim.Hist. baselines

- **CleanTable:** A cleaning activity that involves challenging cloth manipulation and fluids for table cleaning. Action space: `navigate`, `pick`, `dip`, `wipe`. The goal is to cleaning a table with a soaked cloth (see Fig. A.9 (c)).

Each activity takes place in a different B1K scene, `Rs_int`, `mockup_apr`, and `restaurant_hotel`.

F.3 Training details

Learning objectives. For RL-Prim. and RL-Prim.Hist., we use an on-policy reinforcement learning algorithm Proximal Policy Optimization (PPO). θ is the policy parameter, $\hat{E}_t$ denotes the empirical expectation over timesteps. r_t is the ratio of the probability under the new and old policies, respectively. $\hat{A}_t$ is the estimated advantage at time t . ϵ is the clipping hyperparameter. The objective function of PPO is shown in Equation 1:

$$C^{CLIP}(\theta) = \hat{E}_t[\min r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t]. \quad (1)$$

For RL-VMC, we use Soft Actor Critic (SAC) algorithm. The objective function is shown in Equation 2:

$$\pi^* = \arg \max_{\pi} E_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t (R(s_t, a_t, s_{t+1}) + \alpha H(\pi(\cdot|s_t))) \right], \quad (2)$$

where α is the coefficient that determines the weight of two terms and H denotes the entropy.

Reward function. We use the success signal provided by the BDDL activity definition as the reward function for training the policy in all methods. For instance, in the StoreDecorations task, the BDDL task goal definition is `Forall(decoration){Inside(decoration, cabinet)}`. The agent receive a positive signal if and only if one of the decoration is physically placed inside the drawer of the cabinet. This is a challenging sparse reward setting since the agent needs to plan multiple steps to reach the final goal without any intermediate subgoal rewards (e.g. push open the drawer must take place before pick and place the decoration, but no reward signal will be given to the push open action).

Hyperparameters. The hyperparameters for SAC and PPO in the baselines are summarized in Table A.11 and Table A.12. Both SAC and PPO are trained for 30,000 time steps. We train with seeds 0, 1, 2 and evaluate with seed 0.

Computation. For each run of our experiments, we use either a single Nvidia GeForce RTX 2080 Ti or a single Nvidia RTX A-6000 GPU, together with Intel Xeon CPU, and 40GB of RAM. During training, the GPU memory usage is around 9GB. Depending on which task, the total training iterations range between 10k to 25k, and the total wall-clock training time range between 3.75 to 7.5 hours.

F.4 Action Primitives

BEHAVIOR-1K activities are long-horizon which require hundreds if not thousands of environment steps (low-level robot control signals) to be completed. A way to overcome this challenge (e.g., reinforcement learning) is to modify the original action space with a set of *action primitives*, i.e., time-extended actions that correspond to multiple low-level commands and that achieve some expected outcome, such as grasping an object or navigating to a location. For our baselines, we defined six of these primitives, and implemented them using a sampling-based motion planner.

All the primitives are compositional: collision-free paths of the entire robot to navigate to a location, collision-free paths of the robot’s arm to reach a 6D_pose with the end-effector, or a predefined arm joint configuration, trajectories where the end-effector follows a line in Cartesian space, possibly colliding/interacting with the environment, and sequences of actions to open/close the robot’s gripper while holding the position. All manipulation primitives start with a trajectory to move the arm from a tucked configuration (folded arm) to an untucked configuration to enable subsequent arm interaction, and end with the inverse motion, from untucked to tucked configuration, to allow subsequent collision-free navigation. The action primitives take as parameter an object to apply the action on; we assume access to the 3D position of the objects to obtain the necessary parameters to query the motion planner. This procedure is replicated on the real robot, where we combine detections from YOLO [70] with information from the depth map of the RGB-D images to obtain the parameters (see Sec. G). For the navigation actions, we assume a set of known relevant locations to navigate to (next to the rectangles in Fig. A.9)

The primitives we used in our experiments can be seen in Fig. A.10 and include:

- a) navigate: Collision-free trajectory of the entire robot to a location.
- b) pick: Composition of 1) a trajectory to a pre-grasp 6D_pose above the object to pick, 2) a line trajectory in Cartesian space down towards the object, interrupted when there is contact, 3) a closing action, 4) a first retracting trajectory following a line upwards, and 5) a second retracting trajectory to reach the untucked joint configuration. The primitive only executes if the robot is not currently picking another object.
- c) place: Composition of 1) a trajectory to a pre-place 6D_pose above the object to place the grasped object on, 2) an opening action, 3) a second retracting trajectory to reach the untucked joint configuration. The primitive only executes if the robot is currently holding an object.
- d) push: Composition of 1) a trajectory to a pre-push 6D_pose above the object to push-open, 2) a line trajectory in Cartesian space down towards the object, interrupted if there is contact, 3) a line trajectory in Cartesian space to push-open the object, e.g., towards the robot, 4) a first retracting trajectory following a line upwards, and 5) a second retracting trajectory to reach the untucked joint configuration.
- e) dip: Composition of 1) a trajectory to a pre-dip 6D_pose above the object to dip into, 2) a line trajectory in Cartesian space down towards the object to dip, 3) a line trajectory in Cartesian space upwards, and 4) a retracting trajectory to reach the initial joint configuration. The primitive only executes if the robot is currently holding an object.
- f) wipe: Composition of 1) a trajectory to a pre-wipe 6D_pose above the object to wipe, 2) a line trajectory in Cartesian space down towards the object to be wiped, interrupted when there is contact, 3) a line trajectory in Cartesian space horizontally to wipe left-right, 4) a line trajectory in Cartesian space horizontally to wipe towards the robot, 4) a first retracting trajectory following a line upwards, and 5) a second retracting trajectory to reach the untucked joint configuration. The primitive only executes if the robot is currently holding an object.

F.5 Additional Metrics: success score Q

The success score Q [27] for each task is reported in Table A.13. We observe the same trend in Q as in the success rate. This is another indication of the effectiveness of our approach.

Method	Policy Features		Success score Q		
	Primitives	History	StoreDecoration	CollectTrash	CleanTable
RL-VMC	✗	✗	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
RL-Prim.	✓	✗	0.50 ± 0.05	0.49 ± 0.04	0.77 ± 0.08
RL-Prim.Hist.	✓	✓	0.59 ± 0.03	0.68 ± 0.02	0.88 ± 0.02

Table A.13: Performance of three baseline methods, in terms of the success score Q.

G Details of the Real-World Setup

Scene. Our real-world experiments take place in a mockup studio-apartment in our lab that includes a bedroom, living room, and dining area. This real-world scene was modeled with a digital counterpart in simulation for BEHAVIOR-1K: the `mockup_apt` scene. Fig. 5 depicts both real and digital twin side-by-side. The digital model has been created by first, scanning the real-world apartment using a phone and commercial software [83], then, replacing and projecting the texture of walls, floors, and ceilings, and finally, replacing the existing objects with 3D models from our BEHAVIOR-1K DATASET. This process should minimize the effect caused by differences between the real world and the 3D model. The simulated and the real robot use the same 2D map of the scene for localization. In this way, 2D locations in the real world correspond to the same 2D locations in simulation.

Robot platform. We use in our experiments a Tiago++ model from PAL Robotics, with an omnidirectional base, two 7-degrees-of-freedom arms with parallel-yaw grippers, a 1-degree-of-freedom prismatic torso, two SICK LiDAR sensors (back and front of the base), and an ASUS Xtion RGB-D camera mounted on the robot’s head, which can be controlled in yaw and pitch. All sensors and actuators are connected through the Robot Operating System, ROS [84]. The code runs on a laptop with an Nvidia GTX 1070 that sends the commands to the onboard robot computer to be executed.

Action primitives on the real robot. We implemented a version of the action primitives `navigate`, `pick` and `place` on the real robot similar to their implementation in OMNIGIBSON. For navigation, we use a 2D sampling-based motion planner on the 2D map we use for localization and that has been created from the 3D model of the apartment. For manipulation in the real world, we need to obtain the parameters necessary to plan the manipulation primitives (`pick` and `place`) from the sensor signals. To do that, we first obtain detections from a YOLO v3 [70] object detector on the RGB images from the robot’s camera. If the robot attempts to execute an action primitive on an object that has not been detected, we return failure (*Object Detection Failure* in our analysis). If the object to manipulate has been detected, we query the pixels in the depth map corresponding to the 21×21 window at the center of the detection bounding box, and use this information to compute the centroid of the corresponding 3D points. This location will be used as an object location for grasping or placing. This information is enough to create a sequence of paths using a sampling-based motion planner (RRT [62]) similar to the ones used in simulation. The motion planner uses a voxel representation of the scene obtained from the depth sensor.

Experimental setup. We randomized three parameters in our real robot experiments: the initial location of the robot (chosen from the three possible activity locations), the positions of the objects on the table, and the orientations of the objects (upright or lying down). The experiment runs cover uniformly these parameters. Additionally, for vision-based policies, we evaluated two lighting conditions: full lighting (including the ceiling) and only lamps. Failures are defined as follows: Motion planning failures occurred when the robot was unable to plan an end effector trajectory to complete the action primitive. This includes failures caused by navigation noise, where after navigating to the task location, the robot was too far away from the target object to execute the `pick` or `place` action. Grasping failures include errors that occurred during the execution of the grasp such as pushing the object off the table while attempting a grasp or losing the object because it slips out, as well as the robot dropping the object after grasping it. Placing failures include the robot dropping the object in the wrong location. Object detection failures primarily resulted from our object detector, YOLO v3 [70], failing to detect the object to interact with; a second, less common, object detection failure corresponds to the depth camera returning invalid measurements for the detected object. Finally, we consider policy failures when the policy selects the same invalid action three times in a row, e.g., requesting to navigate to the bin when it is already in front of the bin. The main policy error corresponds to the policy repeatedly choosing the same action primitive. We observed empirically that, even though the policy selection presents some small randomness when the policy requests the same invalid action three times in a row, most subsequent calls will be also the same invalid action and we decide to terminate early.

G.1 Further Characterization of the Sim-Real Gap

We performed additional experiments to characterize the gap between simulation and the real world. In our experiments, we moved the robot to different locations in the `mockup_apr` scene and collected real sensor signals: RGB images, depth maps, and LiDAR measurements. We then moved the robot in simulation to the same locations and collected virtual sensor signals. Some of the images are depicted in Fig. A.12. We observe that, thanks to our highly realistic models, OMNIGIBSON provides high-fidelity sensor signals that approximate the real-world ones. However, some of the sources of noise in the real sensors are not modeled currently in OMNIGIBSON, contributing to a sim-real gap, e.g., the poor dynamic range of the RGB camera, or the “shadow” effects in depth maps due to the projected light mechanism. This analysis indicates possible avenues to further close the sensor gap between simulation and the real world, e.g., by including sensor noise models in OMNIGIBSON or by leveraging sim-to-real techniques (e.g., domain randomization, system identification).

G.2 Additional Experiments

In addition to RL-Prim.Hist., we also evaluated RL-VMC and RL-Prim.Hist. in the real world and observed a similar trend of performance in the real world as in simulation: $\text{RL-VMC} < \text{RL-Prim.} < \text{RL-Prim.Hist.}$. RL-VMC still achieves zero success because of sparse reward and exploration difficulty during training. RL-Prim. has worse performance than RL-Prim.Hist.: on average, the robot with RL-Prim.Hist. successfully places 0.64 cups/bottles, whereas the one with RL-Prim. places 0. Qualitatively, RL-Prim. tends to get stuck in a repetitive action loop because the agent is unaware of its action history. This preliminary result offers us some confidence that OMNIGIBSON can be a reliable test bed for future sim-to-real robotics research.

H Ethical Statement

BEHAVIOR-1K aims to drive embodied AI solutions that fulfill human need. A primary ethical consideration of BEHAVIOR-1K is therefore the method used to determine such need. To understand which activities would be useful for humans, we survey 1,461 respondents on Amazon Mechanical Turk and collected 50 responses for each activity, aiming for a clear consensus signal. Though this is a large group, the results are still biased by the fact that the researchers, annotators, and data providers only represent a small population relative to potential users of such technology. We plan to address these biases by making BEHAVIOR-1K DATASET open-source and invite a wider community to contribute to its knowledge base.

Furthermore, compared to the U.S. population, the demographic representation in our survey skews white, male, non-disability status, mid-five figure incomes, and 30-40 age range with more representation on the higher side than the lower side of that range—closer to the demographics of Mechanical Turk workers. This also biases the survey population toward certain responses, creating potential ethical limitations.

Specifically this bias may affect the survey’s representation of people who would be affected most by autonomous agents [85]. We therefore asked several explicit questions such as “Do you do household work for a living?”, “If you do household work for a living, would you benefit from assistance?”, “Do you pay for someone else to do household work for you?”. Future work involves using these questions to direct benchmark development.

Listing 1: BakingSugarCookies

```
(define (problem baking_sugar_cookies -0)
  (:domain omnigibson)

  (:objects
    flour.n.01_1 - flour.n.01
    granulated_sugar.n.01_1 - granulated_sugar.n.01
    raw_egg.n.01_1 - raw_egg.n.01
    vanilla.n.02_1 - vanilla.n.02
    melted__butter.n.01_1 - melted__butter.n.01
    sodium_carbonate.n.01_1 - sodium_carbonate.n.01
    baking_powder.n.01_1 - baking_powder.n.01
    electric_mixer.n.01_1 - electric_mixer.n.01
    mixing_bowl.n.01_1 - mixing_bowl.n.01
    sugar_cookie.n.01_1 sugar_cookie.n.01_2 sugar_cookie.n.01_3
    sugar_cookie.n.01_4 sugar_cookie.n.01_5 sugar_cookie.n.01_6 - sugar_cookie.n.01
    oven.n.01_1 - oven.n.01
    cookie_sheet.n.01_1 - cookie_sheet.n.01
    flour__sack.n.01_1 - flour__sack.n.01
    sugar__sack.n.01_1 - sugar__sack.n.01
    sodium_carbonate__jar.n.01_1 - sodium_carbonate__jar.n.01
    baking_powder__jar.n.01_1 - baking_powder__jar.n.01
    mason_jar.n.01_1 - mason_jar.n.01
    countertop.n.01_1 countertop.n.01_2 - countertop.n.01
    vanilla__bottle.n.01_1 - vanilla__bottle.n.01
    salt.n.02_1 - salt.n.02
    salt__shaker.n.01_1 - salt__shaker.n.01
    bowl.n.01_1 - bowl.n.01
    electric_refrigerator.n.01_1 - electric_refrigerator.n.01
    tablespoon.n.02_1 - tablespoon.n.02
    floor.n.01_1 - floor.n.01
    agent.n.01_1 - agent.n.01
  )

  (:init
    (filled flour__sack.n.01_1 flour.n.01_1)
    (ontop flour__sack.n.01_1 countertop.n.01_1)
    (insource salt__shaker.n.01_1 salt.n.02_1)
    (ontop salt__shaker.n.01_1 countertop.n.01_1)
    (ontop tablespoon.n.02_1 countertop.n.01_1)
    (filled sugar__sack.n.01_1 granulated_sugar.n.01_1)
    (ontop sugar__sack.n.01_1 countertop.n.01_1)
    (inside raw_egg.n.01_1 bowl.n.01_1)
    (insource vanilla__bottle.n.01_1 vanilla.n.02_1)
    (ontop vanilla__bottle.n.01_1 countertop.n.01_1)
    (filled mason_jar.n.01_1 melted__butter.n.01_1)
    (ontop mason_jar.n.01_1 countertop.n.01_1)
    (filled sodium_carbonate__jar.n.01_1 sodium_carbonate.n.01_1)
    (ontop sodium_carbonate__jar.n.01_1 countertop.n.01_2)
    (filled baking_powder__jar.n.01_1 baking_powder.n.01_1)
    (ontop baking_powder__jar.n.01_1 countertop.n.01_2)
    (ontop electric_mixer.n.01_1 countertop.n.01_2)
    (attached mixing_bowl.n.01_1 electric_mixer.n.01_1)
    (inroom oven.n.01_1 kitchen)
    (inroom countertop.n.01_1 kitchen)
    (inroom countertop.n.01_2 kitchen)
    (inside bowl.n.01_1 electric_refrigerator.n.01_1)
    (inroom electric_refrigerator.n.01_1 kitchen)
    (ontop cookie_sheet.n.01_1 countertop.n.01_2)
    (inroom floor.n.01_1 kitchen)
    (ontop agent.n.01_1 floor.n.01_1)
    (future sugar_cookie.n.01_4)
    (future sugar_cookie.n.01_1)
    (future sugar_cookie.n.01_2)
    (future sugar_cookie.n.01_6)
    (future sugar_cookie.n.01_3)
    (future sugar_cookie.n.01_5)
  )

  (:goal
    (and
      (real ?sugar_cookie.n.01_1)
      (real ?sugar_cookie.n.01_2)
      (real ?sugar_cookie.n.01_3)
      (real ?sugar_cookie.n.01_4)
      (real ?sugar_cookie.n.01_5)
      (real ?sugar_cookie.n.01_6)
    )
    (forall
      (?sugar_cookie.n.01 - sugar_cookie.n.01)
      (and
        (cooked ?sugar_cookie.n.01)
        (ontop ?sugar_cookie.n.01 ?cookie_sheet.n.01_1)
      )
    )
  )
)
```

Listing 2: CleanYourLaundryRoom

```
(define (problem clean_your_laundry_room_1)
  (:domain omnigibson)

  (:objects
    rag.n.01_1 - rag.n.01
    dryer.n.01_1 - dryer.n.01
    water.n.06_1 - water.n.06
    vinegar.n.01_1 - vinegar.n.01
    washer.n.03_1 - washer.n.03
    dust.n.01_1 - dust.n.01
    mold.n.05_1 - mold.n.05
    floor.n.01_1 - floor.n.01
    agent.n.01_1 - agent.n.01
  )

  (:init
    (ontop rag.n.01_1 dryer.n.01_1)
    (not
      (covered water.n.06_1 rag.n.01_1)
    )
    (empty vinegar.n.01_1 washer.n.03_1)
    (covered dust.n.01_1 dryer.n.01_1)
    (covered mold.n.05_1 washer.n.03_1)
    (onfloor agent.n.01_1 floor.n.01_1)
    (filled water.n.06_1 bottle.n.01_1)
    (onfloor bottle.n.01_1 floor.n.01_1)
    (inroom washer.n.03_1 laundry_room)
    (inroom floor.n.01_1 laundry_room)
  )

  (:goal
    (and
      (filled ?vinegar.n.01_1 ?washer.n.03_1)
      (not
        (covered ?dust.n.01_1 ?dryer.n.01_1)
      )
      (not
        (covered ?mold.n.05_1 ?washer.n.03_1)
      )
    )
  )
)
```

Listing 3: CleanTheBottomOfAnIron

```
(define (problem clean_the_bottom_of_an_iron-0)
  (:domain omnigibson)

  (:objects
    tarnish.n.02_1 - tarnish.n.02
    iron.n.04_1 - iron.n.04
    ironing_board.n.01_1 - ironing_board.n.01
    emery_paper.n.01_1 - emery_paper.n.01
    countertop.n.01_1 - countertop.n.01
    newspaper.n.03_1 - newspaper.n.03
    floor.n.01_1 - floor.n.01
    agent.n.01_1 - agent.n.01
  )

  (:init
    (covered iron.n.04_1 tarnish.n.02_1)
    (toggled_on iron.n.04_1)
    (ontop iron.n.04_1 ironing_board.n.01_1)
    (ontop ironing_board.n.01_1 floor.n.01_1)
    (ontop emery_paper.n.01_1 countertop.n.01_1)
    (ontop newspaper.n.03_1 ironing_board.n.01_1)
    (inroom countertop.n.01_1 utility_room)
    (inroom floor.n.01_1 utility_room)
    (ontop agent.n.01_1 floor.n.01_1)
  )

  (:goal
    (and
      (not
        (covered ?iron.n.04_1 ?tarnish.n.02_1)
      )
    )
  )
)
```

Listing 4: PackingLunch

```

(define
  (problem packing_lunches_1)
  (:domain igibson)

  (:objects
    shelf.n.01_1 - shelf.n.01
    water.n.06_1 - water.n.06
    countertop.n.01_1 - countertop.n.01
    apple.n.01_1 - apple.n.01
    electric_refrigerator.n.01_1 -
      electric_refrigerator.n.01
    hamburger.n.01_1 - hamburger.n.01
    basket.n.01_1 - basket.n.01
  )

  (:init
    (ontop water.n.06_1 countertop.n.01_1)
    (inside apple.n.01_1
      electric_refrigerator.n.01_1)
    (inside hamburger.n.01_1
      electric_refrigerator.n.01_1)
    (ontop basket.n.01_1 countertop.n.01_1)
    (inroom countertop.n.01_1 kitchen)
    (inroom electric_refrigerator.n.01_1
      kitchen)
    (inroom shelf.n.01_1 kitchen)
  )

  (:goal
    (and
      (for_n_pairs
        (1)
        (?hamburger.n.01 - hamburger.n.01)
        (?basket.n.01 - basket.n.01)
        (inside ?hamburger.n.01 ?basket.n.01)
      )
      (for_n_pairs
        (1)
        (?basket.n.01 - basket.n.01)
        (?water.n.06 - water.n.06)
        (inside ?water.n.06 ?basket.n.01)
      )
      (for_n_pairs
        (1)
        (?basket.n.01 - basket.n.01)
        (?apple.n.01 - apple.n.01)
        (inside ?apple.n.01 ?basket.n.01)
      )
      (forall
        (?basket.n.01 - basket.n.01)
        (ontop ?basket.n.01 ?countertop.n.01_1)
      )
    )
  )
)

```

Listing 5: ServingHorsDoeuvres

```

(define
  (problem serving_hors_d_oeuvres_1)
  (:domain igibson)

  (:objects
    tray.n.01_1 tray.n.01_2 - tray.n.01
    countertop.n.01_1 - countertop.n.01
    oven.n.01_1 - oven.n.01
    sausage.n.01_1 sausage.n.01_2 - sausage.n.01
    cherry.n.03_1 cherry.n.03_2 - cherry.n.03
    electric_refrigerator.n.01_1 -
      electric_refrigerator.n.01
  )

  (:init
    (ontop tray.n.01_1 countertop.n.01_1)
    (ontop tray.n.01_2 countertop.n.01_1)
    (inside sausage.n.01_1 oven.n.01_1)
    (inside sausage.n.01_2 oven.n.01_1)
    (inside cherry.n.03_1
      electric_refrigerator.n.01_1)
    (inside cherry.n.03_2
      electric_refrigerator.n.01_1)
    (inroom oven.n.01_1 kitchen)
    (inroom electric_refrigerator.n.01_1
      kitchen)
    (inroom countertop.n.01_1 kitchen)
  )

  (:goal
    (and
      (exists
        (?tray.n.01 - tray.n.01)
        (and
          (forall
            (?sausage.n.01 - sausage.n.01)
            (ontop ?sausage.n.01 ?tray.n.01)
          )
          (forall
            (?cherry.n.03 - cherry.n.03)
            (not
              (ontop ?cherry.n.03 ?tray.n.01)
            )
          )
        )
      )
      (exists
        (?tray.n.01 - tray.n.01)
        (and
          (forall
            (?cherry.n.03 - cherry.n.03)
            (ontop ?cherry.n.03 ?tray.n.01)
          )
          (forall
            (?sausage.n.01 - sausage.n.01)
            (not
              (ontop ?sausage.n.01 ?tray.n.01)
            )
          )
        )
      )
    )
  )
)

```



Figure A.3: Part 1 of a scene collage showing views of all 50 scenes.

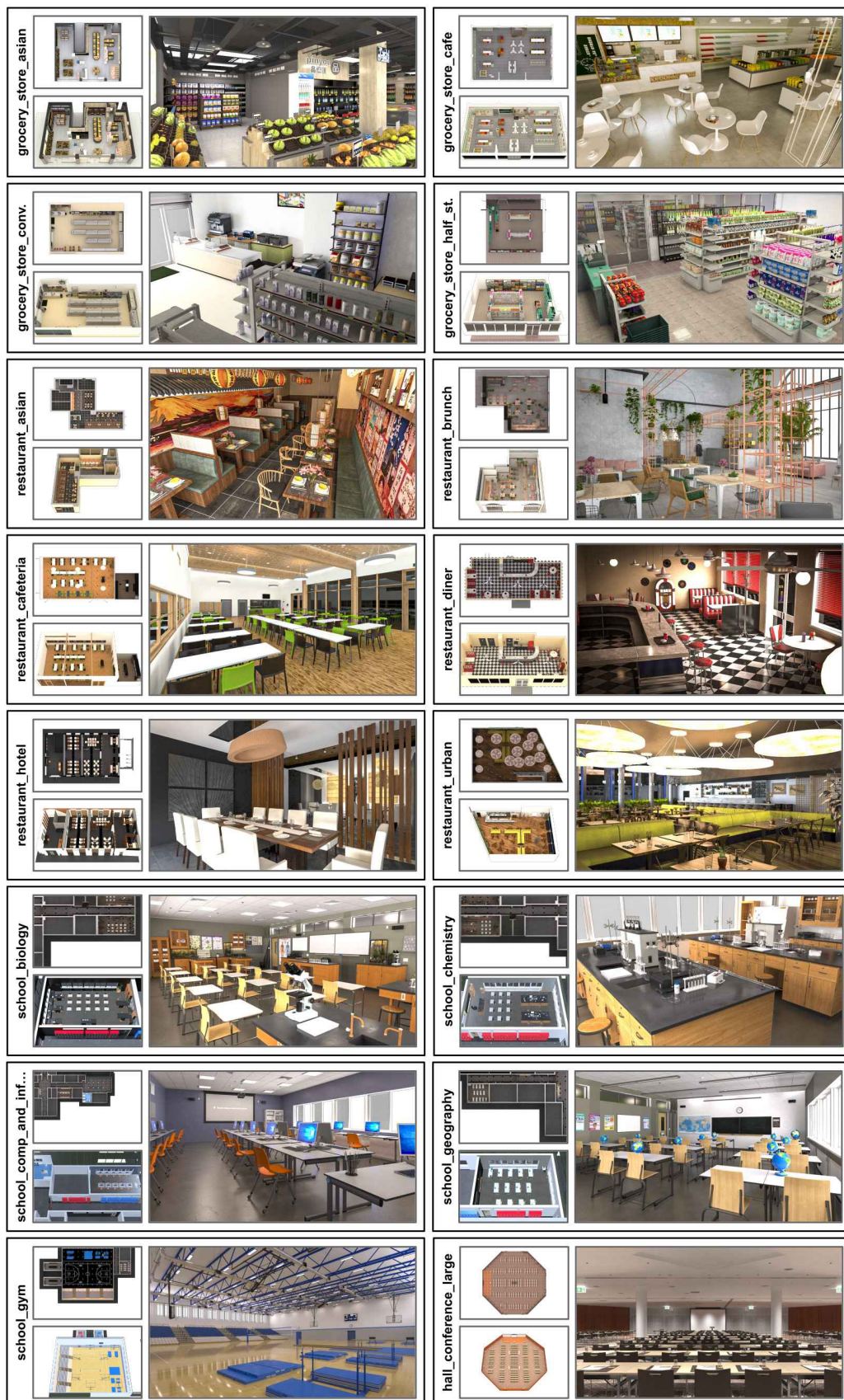


Figure A.4: Part 2 of a scene collage showing views of all 50 scenes.

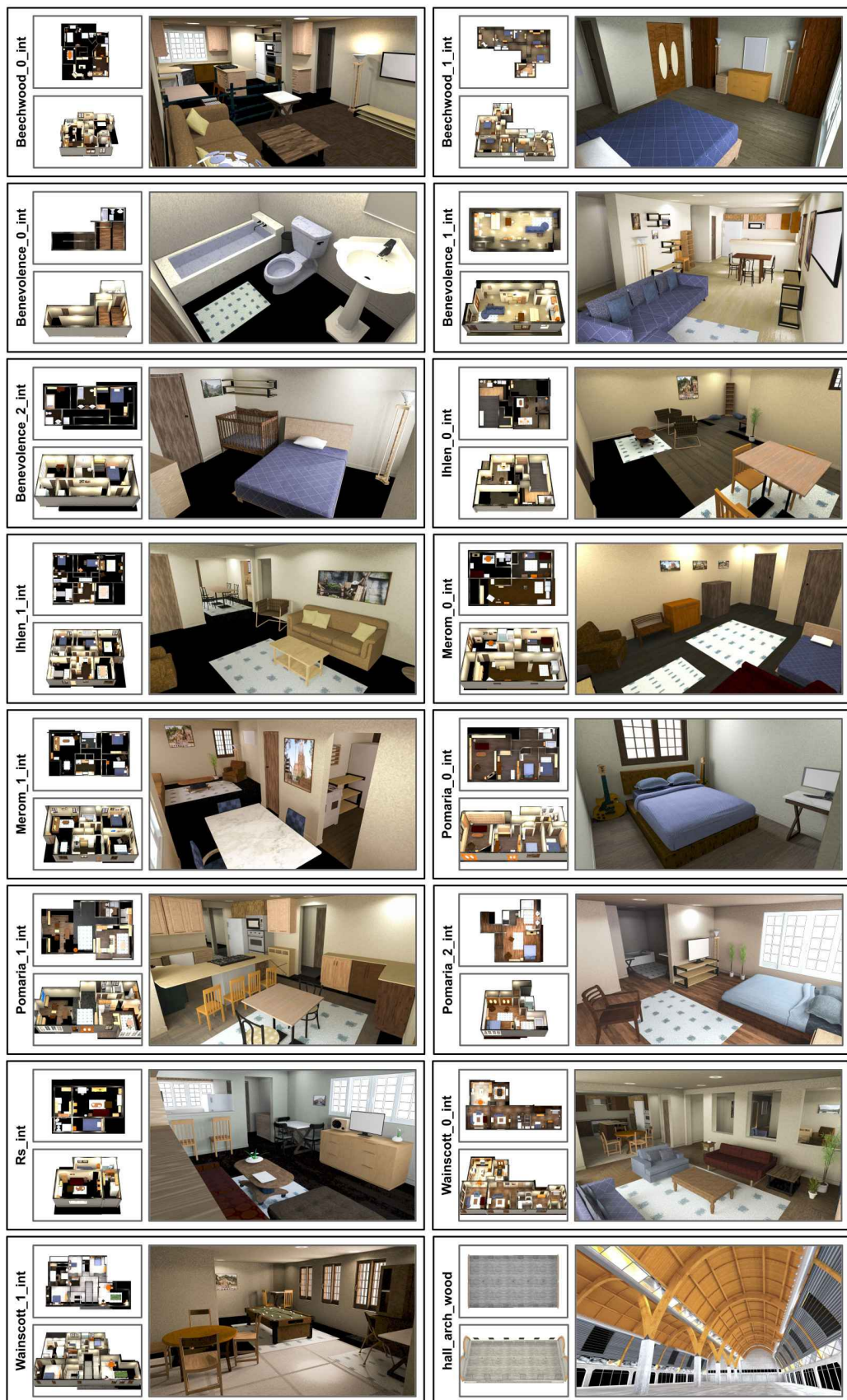


Figure A.5: Part 3 of a scene collage showing views of all 50 scenes.

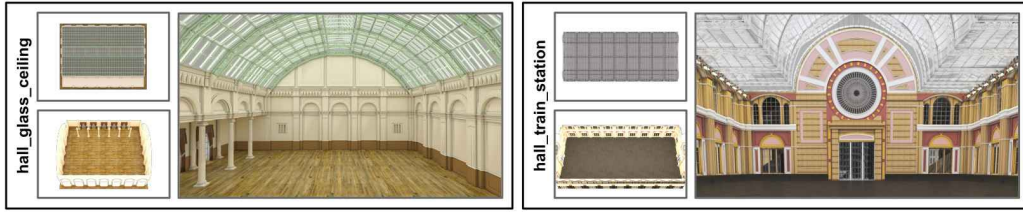


Figure A.6: Part 4 of a scene collage showing views of all 50 scenes.

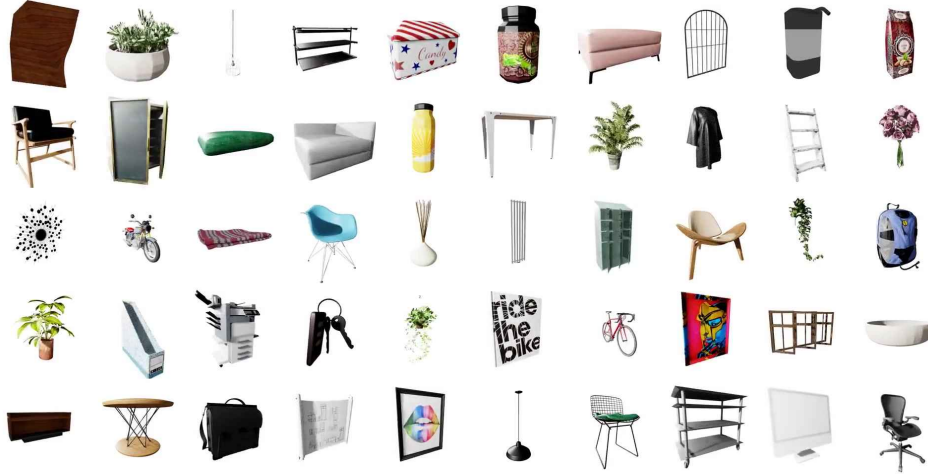


Figure A.7: A collage of objects included in the BEHAVIOR-1K DATASET. These objects highlight key features of OMNIGIBSON such as transparency, articulation, heat simulation, and lighting.

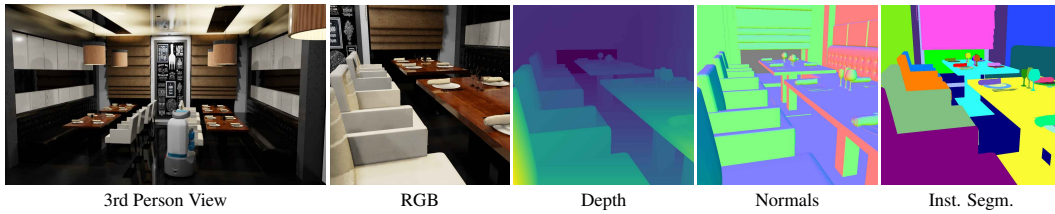


Figure A.8: **Visual modalities provided by OMNIGIBSON.** The robot observes the scene (left, 3rd person view) and obtains visual observations from its onboard sensors including RGB images, depth, normals, and instance segmentation. Rendered RGB images are highly realistic, and, combined with the diverse set of visual observations, can be used to train visual policies.

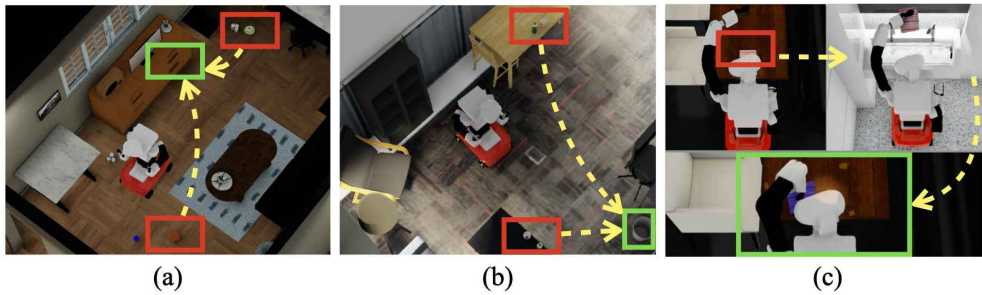


Figure A.9: **Activities used in our evaluation.** From left to right: (a). StoreDecoration, (b). CollectTrash and (c). CleanTable. The rectangles indicate relevant locations for the activities, e.g., locations with objects to grasp (*red rectangles*) or to use/place them (*green rectangles*). Even though these activities are some of the most simple in BEHAVIOR-1K, they are still very long-horizon, requiring hundreds of low-level commands or the correct concatenation of multiple action primitives.

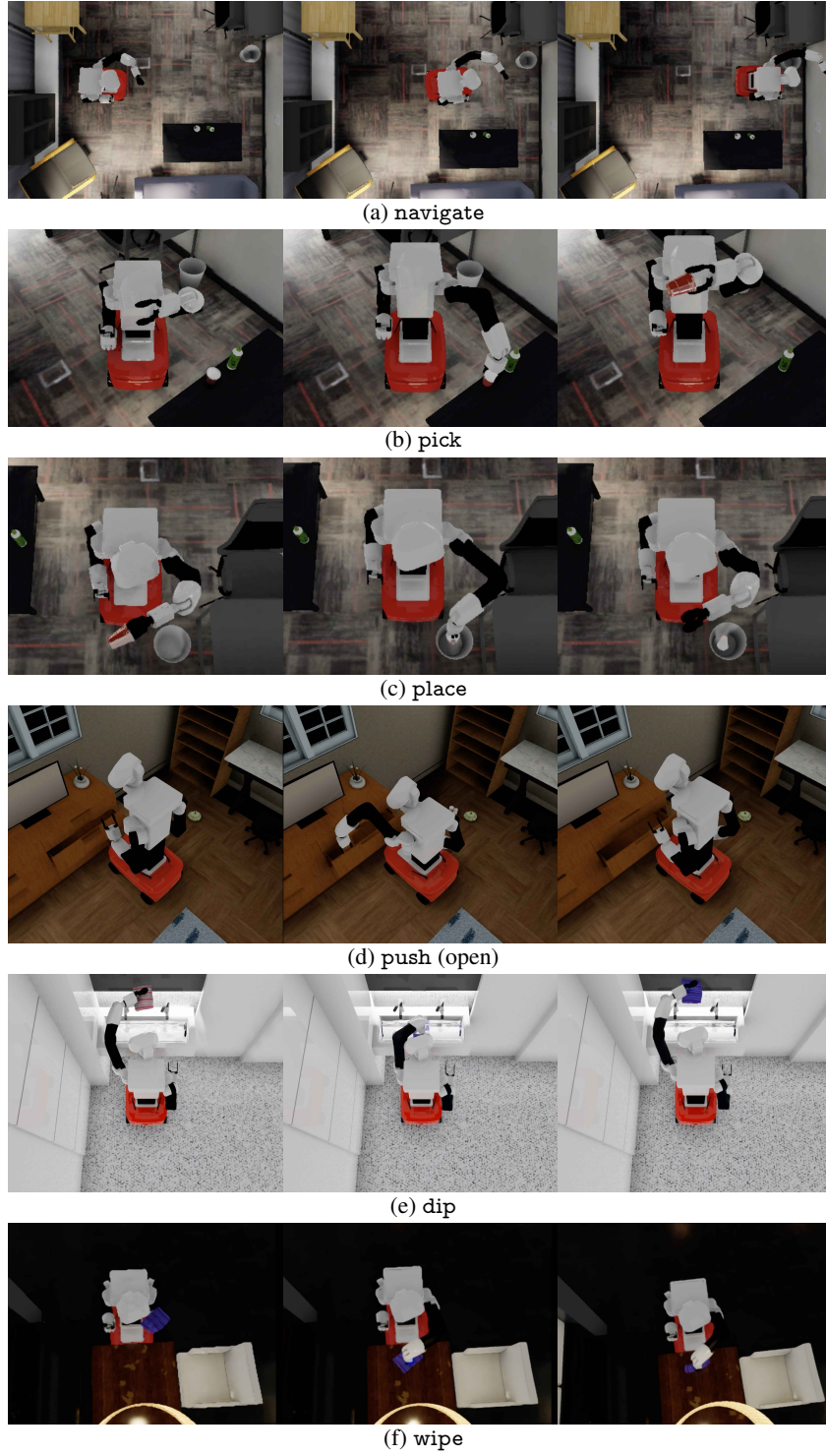


Figure A.10: Overview of all the high-level action primitives with the pre-conditions (left), post-conditions (right), and intermediate states (middle) during execution.

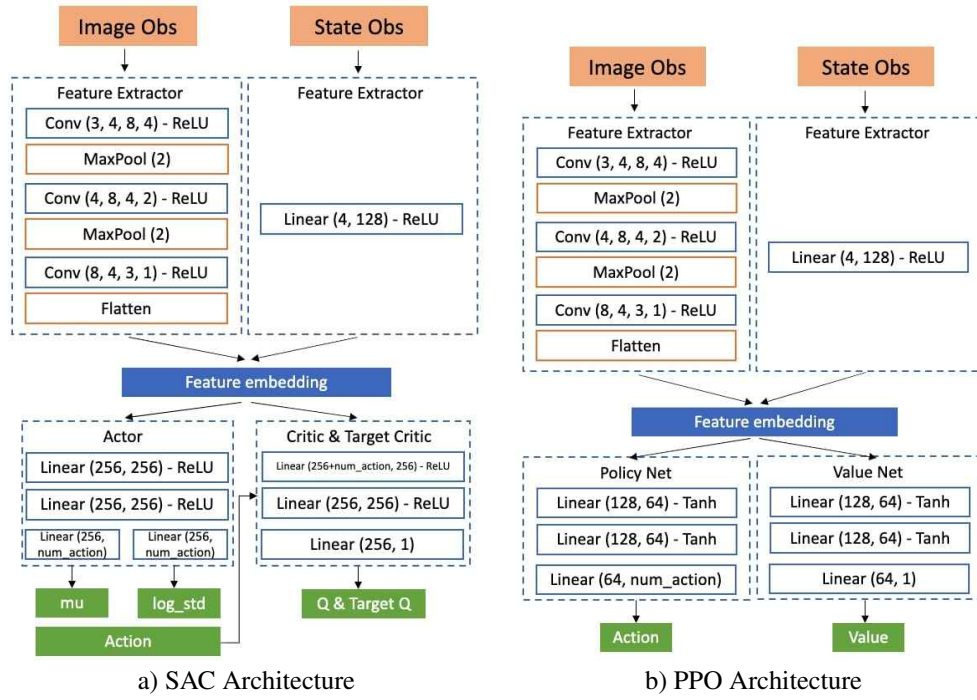


Figure A.11: Policy architecture of SAC (RL-VMC) and PPO (RL-Prim. and RL-Prim.Hist.). The policy maps egocentric visual observations and proprioceptive information into an action that controls the robot base, arm, and gripper. PPO outputs a discrete high-level action primitive while SAC outputs a continuous low-level joint control directly.

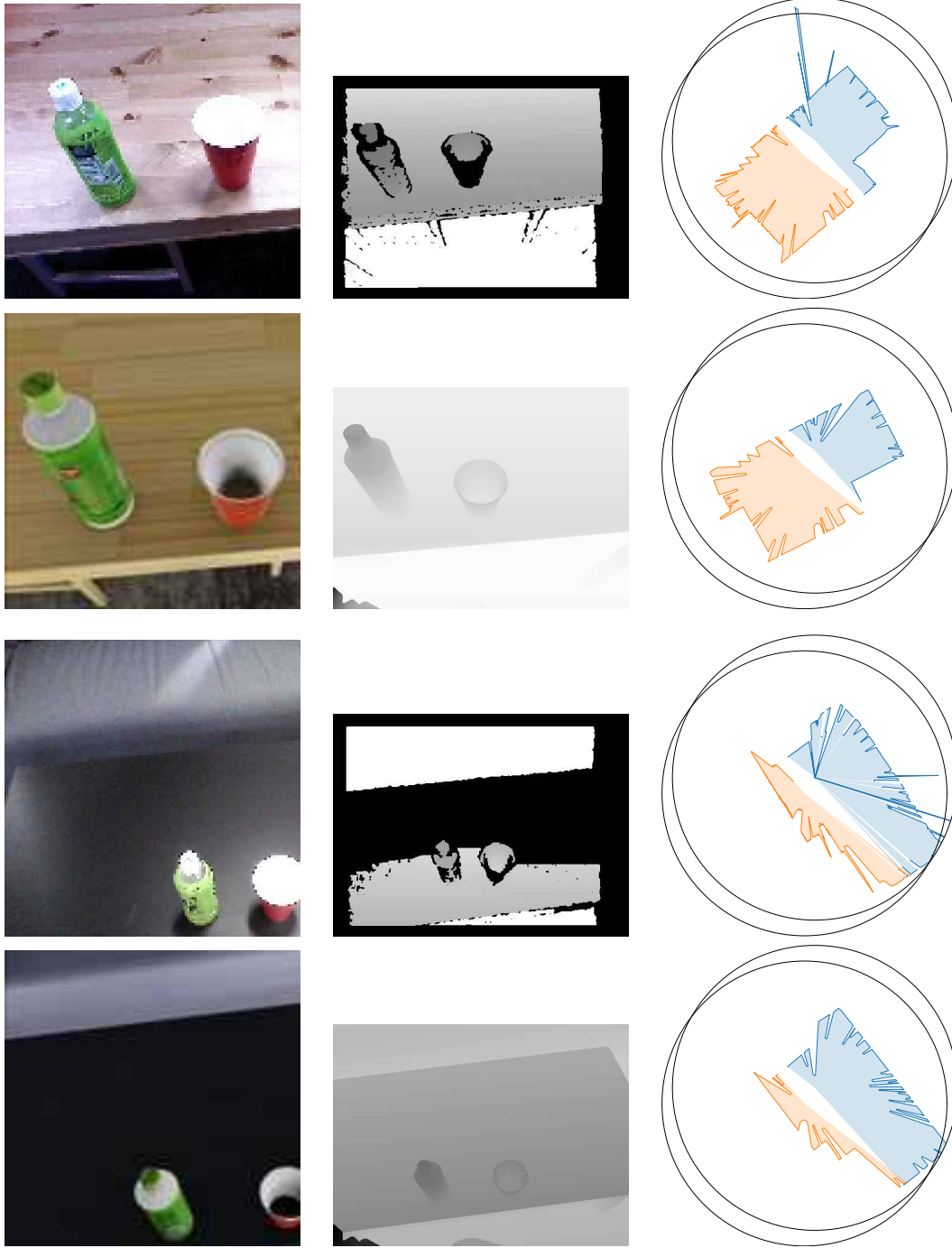


Figure A.12: **Comparison between real and simulated sensor signals.** RGB, Depth and LiDAR signals from the real-world sensors (first and third rows) and from the simulated sensors (second and fourth rows) at two different locations of the `mockup_apr` scene (location 1: first two rows, location 2: last two rows). We observed a smaller sensor sim-real gap for LiDAR than for RGB and Depth: RGB is heavily influenced by lighting conditions and camera settings while Depth has difficulty in capturing reflective surfaces.